Autoservis Reactivation SaaS — Master Build Specification

Sep 25, 2026 · @Ondřej Luxa

0. Challenged assumptions (read first)

The idea is sound, but eleven assumptions in the brief would hurt you if built as written. Each one below changes the spec that follows. Legal points are a starting checklist, not legal advice; have a Czech lawyer review sections 0.1 and 21 before the first real send.

0.1 Marketing permission is the legal gate, not an optional field

SMS and email marketing in Czechia falls under Act No. 480/2004 Coll., §7, on top of GDPR. Sending requires prior consent, or the "existing customer" exception: the contact was obtained when selling a service, the message promotes the shop's own similar services, and the customer could refuse at collection and can refuse in every message. Most shop exports carry no consent record at all.

- Every import must record the legal basis per customer: `consent`, `existing_customer`, or `unknown`.
- `unknown` customers are excluded from sending by default.
- The shop owner confirms the legal basis in writing (a checkbox plus timestamp in the import wizard, backed by the DPA).
- Very old contacts (5+ years) are legally weaker under the existing-customer exception. Default the maximum lookback to 4 years; make it configurable.
- Every message carries an opt-out. No exceptions, including follow-ups.
- The shop is the data controller; you are the processor. A signed processing agreement (Art. 28 GDPR, "zpracovatelská smlouva") is required before any import.

0.2 "Recovered revenue" will be overstated without a control group

Some customers would have come back anyway. If you count every returning customer as recovered, the ROI looks great until an owner notices his regulars being "recovered". That destroys trust in the one number the product sells.

- Every campaign holds out a random 10% of eligible recipients who receive nothing (configurable 0–20%).
- The dashboard shows attributed revenue (headline) and, once there is enough data, incremental revenue (treatment return rate minus holdout return rate).

- Attribution uses a fixed window (default 60 days after first message) and one conversion per recipient.

0.3 Manual revenue entry will not happen reliably

A busy service advisor will not mark every returning customer in a second system. Manual marking must exist, but the primary mechanism should be a periodic re-import of the shop's job history (monthly in Phase 2, weekly later). The system then matches new visits to contacted customers by phone, plate or VIN and proposes conversions for confirmation. This turns attribution from a chore into a 2-minute review.

0.4 Do not generate a separate AI message per customer

Per-customer AI generation for 300 recipients produces 300 unreviewable messages, some with invented details. Instead: AI writes one campaign template with placeholders, a human approves it, and deterministic code fills in name, vehicle and shop fields. This is cheaper, faster, testable and legally safer.

0.5 Czech vocative greetings are a trap

"Dobrý den, pane Nováku" requires the 5th case of the surname and a guessed gender. Wrong vocatives ("pane Novák", "paní Novákov") look worse than no name. Default to "Dobrý den," and make the personalized vocative an opt-in with a cached, reviewable vocative per surname (section 15).

0.6 Diacritics triple SMS cost

One diacritic character switches the whole SMS to UCS-2 encoding: 70 characters for a single SMS, 67 per segment when split, versus 160/153 without diacritics. A typical 150-character Czech message costs 3 segments with diacritics and 1 without. The editor must show a live segment counter and offer "odstranit diakritiku" (strip diacritics). Plan limits must count segments, not messages.

0.7 Alphanumeric sender IDs cannot receive replies

If messages come from "AutoXYZ", customers cannot reply, and "STOP" by SMS does not work. Two-way SMS needs a numeric virtual number, which looks less trustworthy. MVP choice: alphanumeric sender with a call-to-action phone number and a one-click opt-out short link. Revisit two-way SMS in Phase 4. The "replied" status is therefore Phase 4 unless you choose a numeric sender.

0.8 Vercel alone cannot run campaigns reliably

Scheduled sends, day-3 follow-ups, provider rate limits and retries do not fit serverless request timeouts. You need a durable job runner (recommended: Inngest; alternative: Trigger.dev). Double-sending a campaign is the worst operational failure this product can have, so every send must be idempotent.

0.9 The unit of reactivation is the vehicle, not the customer

Service intervals belong to cars. One customer can own three cars; a car can change owner. Model `customers`, `vehicles` and `visits` (service events) separately. Import visit-level history when the shop can export it; derive "last regular service" per vehicle from it. A flat customer table with one "last visit" column cannot power the segments in the brief.

0.10 "Potential revenue" of 348,600 CZK is misleading

Eligible × average job assumes a 100% return rate. Show "expected revenue" using a conservative conversion rate (default 6%, later learned per shop), and show the theoretical maximum only as secondary text. Overpromising in the campaign builder creates disappointment in the results screen.

0.11 Other corrections

- **No public sign-up in MVP.** Invite-only. Public sign-up is a Phase 3 feature with its own abuse risks.
- **No "log in as user" impersonation.** Use an audited, banner-marked support view scoped to one company, read-only by default (section 20).
- **No automatic sending in MVP.** Human approval of every campaign until a shop has at least 3 clean campaigns; then auto-send is opt-in per automation.
- **Stripe is not needed early.** Czech B2B clients commonly pay by bank transfer against an invoice. Use Fakturoid recurring invoices for the first 20 clients; add Stripe only when self-service sign-up exists.
- **STK data is not available.** There is no open API giving inspection dates per plate with owner data. STK reminders only work when the shop recorded the date.
- **Revenue is not profit.** ROI on revenue flatters the product. Keep revenue as the headline, but let the shop set an optional margin to show gross profit ROI.

1–3. Product summary, personas, value proposition

1. Executive summary

The product is a managed reactivation service for independent Czech car repair shops, delivered through software. It finds vehicles overdue for recurring work, sends short approved SMS campaigns on the shop's behalf, tracks who comes back, and reports recovered revenue against the subscription price.

The first version is an internal operator tool. One operator (you) runs campaigns for 1–5 shops from a super admin dashboard. Shops see a read-only results dashboard. Self-service, automation and integrations come only after the pilot proves that contacted customers return at a measurably higher rate than the holdout group.

The product has one job: get old customers back and prove how much money that generated. Every feature request is tested against that sentence.

Working product name used in this spec: **Návrat** (replace freely). Code identifiers use the neutral prefix `app`.

2. User personas

Persona	Role in system	Context	Needs	Fears
Operator (you)	`super_admin`	Runs campaigns for 1–20 shops, later oversees 100+	See only exceptions; do a shop's monthly campaign in under 30 minutes	Double-sends, data leaks, a shop churning over unclear results
Pavel, shop owner	`shop_owner`	48, owns a 6-person shop in a mid-sized town, 2,400 historical customers, uses garage software with CSV export	More bookings in quiet months, no extra admin	Paying for "marketing" that does nothing, annoying loyal customers, GDPR trouble
Jana, service advisor	`shop_manager`	Front desk, answers phones, books jobs	Know which callers came from a campaign; mark jobs done in seconds	Another system to log into every day

Persona	Role in system	Context	Needs	Fears
Vehicle owner	none (message recipient)	Receives 1–3 SMS per year at most	Relevant, polite reminder; easy opt-out	Spam, feeling tracked

3. Core value proposition

Czech (primary): "Pomáháme autoservisům vracet zákazníky, kteří se dlouho nevrátili. Vy platíte předplatné, my vám ukážeme, kolik peněz se vrátilo."

For the owner: a revenue-generating employee that costs 4,990 CZK per month and reports its own results.

The proof, shown on every dashboard load:

Metric	Example
Recovered revenue this month	52 300 Kč
Subscription	4 990 Kč
ROI	10,5×
Completed reactivated jobs	11

Note the Czech number format: space as thousands separator, comma as decimal separator, "Kč" suffix. Use `Intl.NumberFormat('cs-CZ', { style: 'currency', currency: 'CZK', maximumFractionDigits: 0 })` everywhere; never hand-format money.

4–5. MVP scope and explicit non-MVP scope

The MVP is Phase 1 (internal tool) plus the minimum of Phase 2 needed for a real pilot. It is done when you can run a real SMS campaign for a real shop and report attributed revenue with a holdout comparison.

4. In the MVP

Area	Included	Notes
Auth	Email + password, password reset, invite-only users, TOTP MFA for super admin	No public sign-up

Area	Included	Notes
Roles	`super_admin` (platform), `shop_owner`, `shop_manager` (company)	Permission table, not hardcoded role checks
Companies	Create/edit shop profile, all fields from the brief, `service_mode` = managed	Self-service mode exists as a flag only
Import	CSV + XLSX wizard: upload, encoding detection, mapping, validation, dedup, preview, commit, history	Customer-level and visit-level imports
Data model	Customers, vehicles, visits, service categories	Vehicle-centric
Lifecycle status	Active / at risk / inactive / reactivated / opted out, recomputed nightly	Thresholds per shop
Segments	Rule builder with 8 presets from the brief + custom rules	JSON rule DSL
Campaigns	Create, AI template, preview per recipient, approval, schedule, send, pause, cancel	One channel: SMS
Follow-ups	Up to 2 follow-up steps with stop conditions	Stops on opt-out, booking, conversion
AI	Template generation + 4 rewrite actions; column-mapping suggestions	Template-based, validated output
SMS	One Czech provider behind an adapter; delivery reports; cost tracking; segment counting	Mock provider in non-production
Opt-out	Short link in every message, one-click opt-out page, suppression list	Survives customer deletion
Conversions	Pipeline Contacted → Booked → Completed, manual revenue entry, visit re-import matching	Holdout group per campaign
Client dashboard	Read-only: recovered revenue, KPI cards, monthly chart, campaigns, recent returns	Mobile responsive
Admin dashboard	Overview, Attention Required, clients list, all campaigns, support view	Exception-first

Area	Included	Notes
Audit log	All events listed in section 23	Append-only
Notifications	In-app + email to operator for the alert types in section 13	Client gets a monthly results email
i18n	Czech UI through a translation layer, English code	English strings optional

5. Explicitly NOT in the MVP

Not building	Why	Earliest phase
Public sign-up, self-service onboarding	You onboard every pilot shop personally	3
Stripe / online payments	Invoice by Fakturoid, bank transfer	3
Plan limits enforcement	Track usage, enforce manually	3
Email campaigns	SMS proves the model; email adds deliverability work	3
Automation engine (auto-enrolment, auto-send)	Needs proven templates and trust	4
AI segment recommendations	Deterministic opportunity cards are enough	4
Two-way SMS, reply inbox, AI reply assistant	Requires numeric sender and inbox UI	4
Booking calendar	Shops already have one	Not planned
Garage software integrations	Need 20+ shops using the same software to justify	5
WhatsApp	Business API cost and template approval overhead	5
A/B testing	Holdout group comes first; variants need volume	4
Churn prediction, CLV models	Rules beat models at this data size	5
Multiple branches, white label, reseller, public API	No demand yet	5

Not building	Why	Earliest phase
Native mobile apps	Responsive web is enough	Not planned
Invoicing, inventory, job cards, accounting	Not the product's job	Never
Google review requests, satisfaction surveys	Different job; distracts from reactivation	5, only if requested

6–7. User flows and information architecture

The product runs on one monthly loop per shop: import fresh history, pick a segment, approve a template, send, confirm returns, report. Everything else supports that loop.

```
flowchart LR
  A[Import job history] --> B[Nightly lifecycle recompute]
  B --> C[Opportunity cards]
  C --> D[Create campaign from segment]
  D --> E[AI template + human edit]
  E --> F[Approval]
  F --> G[Scheduled send + follow-ups]
  G --> H[Returns: manual mark or re-import match]
  H --> I[Attributed revenue + ROI]
  I --> A
```

6. User flows

F1. Onboard a shop (operator, ~20 min)

1. Admin → Klienti → Nový klient. Enter profile, average job value, sender name, opening hours.
2. Upload signed DPA (PDF) and tick "Zpracovatelská smlouva podepsána". Import is blocked until this is done.
3. Invite the owner by email. Owner sets a password; sees an empty dashboard with "Připravujeme vaše data".
4. Configure lifecycle thresholds (defaults prefilled) and service category mapping.

F2. Import data (operator)

1. Klient → Import → upload CSV/XLSX → choose import type (customers, or visits/job history).
2. System detects encoding, delimiter, header row; AI suggests column mapping; operator confirms.

3. Operator declares the legal basis for marketing for this batch, or maps a consent column.
4. Validation report: valid, fixable, invalid, duplicates. Operator fixes mapping or excludes rows.
5. Commit runs as a background job. Result: counts created/updated/skipped, link to the error CSV.

F3. Run a campaign (operator)

1. Klient → Příležitosti shows cards such as "71 zákazníků nebylo 12+ měsíců". Click "Vytvořit kampaň".
2. Campaign builder: segment preselected, eligible count after suppression and frequency caps, expected revenue, holdout 10%.
3. "Vygenerovat zprávu pomocí AI" → template with placeholders → edit → preview for 5 random recipients → segment count and cost.
4. Add follow-ups (optional): day 3, day 7.
5. Pick send time within allowed window. Submit: status `waiting_for_approval` if the shop approves, else `scheduled` after operator confirmation.
6. At send time the job snapshots recipients, dispatches in batches, records delivery reports.

F4. Approve a campaign (shop owner, optional per shop)

Owner gets an email "Kampaň čeká na schválení" → opens a mobile-friendly page with message, recipient count, send time → "Schválit" or "Vrátit s komentářem".

F5. Record a return (shop manager or operator)

1. Customer calls. Advisor searches by phone or plate (global search, top bar).
2. Customer card shows "Oslovený v kampani Servis září · 12. 9." → button "Objednal se" → later "Dokončeno" + job value.
3. Alternatively the monthly visit re-import proposes matches; operator bulk-confirms in Výsledky → Ke kontrole.

F6. Monthly results (automatic)

On the 1st of each month the owner receives an email: recovered revenue, jobs, ROI, one sentence on next month's plan. Operator reviews shops flagged for low conversion.

F7. Opt-out (vehicle owner)

SMS link `https://<short-domain>/o/Xk3P9` → page with shop name and one button "Nepřeji si další zprávy" → confirmation. Phone is added to the shop's suppression list immediately; pending follow-ups are cancelled.

7. Information architecture

Client app (`/app`) — Czech labels shown, English route names.

Menu item	Route	MVP content
Přehled	`/app`	Recovered revenue, KPI cards, chart, campaigns, recent returns, opportunities
Zákazníci	`/app/customers`	Searchable list, customer/vehicle detail, mark booked/completed
Kampaně	`/app/campaigns`	List, detail with results, approval (if enabled)
Automatizace	`/app/automations`	Hidden until Phase 4
Výsledky	`/app/results`	Revenue by month, campaign, segment; conversions to review
Nastavení	`/app/settings`	Profile, users, notification preferences, data & privacy

Admin app (`/admin`) — English or Czech labels, operator's choice.

Menu item	Route	MVP content
Overview	`/admin`	Platform KPIs + Attention Required
Clients	`/admin/companies`	List with health, detail with tabs: profile, import, customers, segments, campaigns, results, audit
Campaigns	`/admin/campaigns`	All campaigns across shops, filter by status
Alerts	`/admin/alerts`	Open/acknowledged/resolved alerts
Billing	`/admin/billing`	Plan, price, invoice status per shop (manual entry in MVP)
System	`/admin/system`	Provider status, job queue health, feature flags, global kill switch, audit search

The client sidebar never shows more than six items. Settings subsections live in tabs, not in the sidebar.

8–10. UI specification, client dashboard, super admin dashboard

Every screen answers one question, and the client dashboard answers "how much money came back" before anything else.

8. Page-by-page UI specification

Global shell

- Left sidebar (collapsible to icons at < 1024 px, bottom sheet menu on mobile), company logo + name top-left.
- Top bar: global search (phone, plate, name; `Ctrl+K`), notification bell, user menu.
- Admin support view shows a persistent amber banner: "Prohlížíte účet: Autoservis XYZ · pouze čtení · Ukončit".
- Empty states always say what happens next, never just "Žádná data".

Page	Purpose	Key elements	Primary action
Přihlášení / Reset hesla	Access	Email, password, forgot link; MFA step for admins	Přihlásit se
Přehled (client)	Show value	See section 9	Zobrazit výsledky
Zákazníci – seznam	Find a customer fast	Search, filters (status, service category, last visit range, eligibility), columns: name, phone, vehicle, last visit, status badge, last contacted	Detail
Zákazník – detail	One screen per customer	Contact + eligibility, vehicles with visit timeline, campaign history, conversion pipeline buttons, notes	Objednal se / Dokončeno
Kampaně – seznam	Status overview	Tabs by status group (Aktivní, Připravené, Dokončené), columns: name, segment, sent, delivered, bookings, revenue	Detail

Page	Purpose	Key elements	Primary action
Kampaň – detail	Performance	Funnel sent → delivered → booked → completed, revenue, holdout comparison, recipient table, message + follow-ups	Pozastavit / Schválit
Kampaň – builder (admin, later client)	Create campaign	4 steps: Segment → Zpráva → Načasování → Kontrola	Odeslat ke schválení
Výsledky	Proof	Revenue by month chart, table per campaign and per segment, "Ke kontrole" queue of proposed matches	Potvrdit shody
Nastavení	Configuration	Tabs: Profil servisu, Uživatelé, Oznámení, Data a soukromí (export, deletion request, DPA download)	Uložit
Opt-out page (public)	Unsubscribe	Shop logo, one sentence, one button, no login	Nepřeji si další zprávy

Customer list for `shop_manager` shows full phone numbers (they need to call back); exports are disabled for managers.

Campaign builder steps

1. **Segment:** pick a preset or saved segment; show eligible count, excluded count with reasons (opted out, no mobile, contacted in last 30 days, no legal basis), holdout slider.
2. **Zpráva:** AI generate button, 4 rewrite chips (Kratší, Přátelštější, Profesionálnější, Znovu), editor with placeholder chips, live segment counter and "Odstranit diakritiku" toggle, preview carousel of 5 real recipients, follow-up steps.
3. **Načasování:** date/time picker constrained to the allowed window, follow-up delays.
4. **Kontrola:** summary card with recipients, total segments, estimated SMS cost, expected revenue, opt-out present check, then submit.

9. Client dashboard specification

Layout top-down, single column on mobile, 12-column grid on desktop.

1. **Hero (full width):** label "Vrácené tržby tento měsíc", value in 56–64 px semibold tabular numerals (e.g. 52 300 Kč), delta line "+18 % oproti minulému měsíci", secondary line "Celkem od začátku spolupráce: 214 900 Kč".
2. **KPI row (5 cards):** Vrácení zákazníci 11 · Objednávky z kampaní 14 · Odeslané zprávy 218 · Předplatné 4 990 Kč · Návratnost 10,5×.
3. **Chart:** "Vrácené tržby po měsících", 12 months, bars for attributed revenue, thin line for subscription cost so ROI is visible without reading numbers.
4. **Two columns:** Aktuální kampaně (max 3, status + progress) · Nedávno vrácení zákazníci (last 5: name, vehicle, job value, campaign).
5. **Příležitosti:** up to 3 cards, e.g. "71 zákazníků u vás nebylo 12+ měsíců · očekávané tržby ~ 18 000 Kč". In managed mode the button reads "Požádat o kampaň" (notifies operator); in self-service "Vytvořit kampaň".

KPI definitions (all for the selected month, company timezone):

KPI	Definition
Vrácené tržby	Sum of `conversions.revenue_amount` with status `completed`, `completed_at` in month, attributed to a campaign
Vrácení zákazníci	Distinct customers with a completed conversion in month
Objednávky	Conversions reaching `booked` or later, `booked_at` in month
Odeslané zprávy	Messages with status `sent` or later, `sent_at` in month
Návratnost (ROI)	Recovered revenue ÷ subscription price for the month; shown as `—` when subscription is 0

If ROI < 1 for two consecutive months, the client sees no warning, but the operator gets an alert. The numbers stay honest; the conversation about them happens in person.

10. Super admin dashboard specification

Overview top row: Aktivní klienti · MRR · Zprávy tento měsíc · Vrácené tržby všech klientů · Aktivní kampaně · Selhání (7 dní).

Attention Required (the core of the admin app). A list of open alerts, grouped by severity, each with the company, a one-line cause, age, and a one-click action. When no alerts are open it shows a single green line: "Vše běží normálně · poslední kontrola 06:00".

Alerts are produced by a scheduled health check (every 15 minutes for sending issues, daily at 06:00 for business rules). Each rule has an idempotent key so the same issue creates one alert, not one per run.

Alert	Rule (defaults, configurable)	Severity	Action
Campaign failed	Campaign in `failed` state or > 5% messages failed at send	Critical	Open campaign
Provider error	SMS API errors > 3 in 10 min or credit below threshold	Critical	System → providers
Abnormal SMS failure rate	Company delivery rate < 90% over last 200 messages	High	Open failed messages
Import needs review	Import in `needs_review` or > 10% invalid rows	High	Open import
Awaiting approval	Campaign `waiting_for_approval` > 48 h	Medium	Remind client
No campaign running	Active company with no send in 35 days	Medium	Open opportunities
Low conversion	Completed campaign with < 2% completed conversions after 45 days (min. 100 recipients)	Medium	Review campaign
ROI below 1	Two consecutive months	Medium	Schedule call
Unreviewed matches	Proposed conversions pending > 7 days	Low	Open review queue
Stale data	No import in 60 days	Low	Request export
Payment overdue	Invoice marked overdue > 14 days	Medium	Billing
Large opportunity	Segment with expected revenue > 30,000 CZK not contacted	Info	Create campaign

Clients list: one row per company with health dot (green/amber/red from open alerts), plan, MRR, recovered revenue this month, ROI, last campaign date, open alerts count. Default sort: red first.

Support view (instead of impersonation): "Otevřít účet klienta" asks for a reason (dropdown + text), opens the client UI in read-only mode with the banner, and writes `admin.support_view_started` to the audit log. Write actions in support view require a second explicit toggle, also logged.

Design system

- **Feel:** Stripe/Linear restraint. White or near-white background, one neutral gray scale, one accent (deep green `#0F7B53` for money and positive states), red only for errors.
- **Type:** Inter (full Czech diacritics support), tabular numerals for all figures. Scale 12/14/16/20/28/40/60.
- **Components:** shadcn/ui primitives only; no custom component library in MVP.
- **Density:** 8 px spacing grid, generous whitespace on dashboards, compact tables (40 px rows).
- **Status badges:** Aktivní (green), Ohrožený (amber), Neaktivní (gray), Vrácený (accent), Odhlášený (outline).
- **Dates:** Czech format `25. 9. 2026`, times `14:30`, relative times "před 3 dny".
- **Avoid:** gradients, glassmorphism, AI sparkle icons everywhere, dark-mode-first styling, more than two font weights per screen.
- **Accessibility:** WCAG 2.1 AA contrast, full keyboard navigation, labels on all inputs.

11. Customer import flow

Imports go through a staging area and never write directly into live tables. A bad file must be impossible to half-import, and a re-import must update rather than duplicate.

Two import types

Type	One row =	Use when	Derives
`customers`	a customer (optionally with one vehicle and last visit)	Shop exports only a contact list	One synthetic visit per row if last visit date + service type exist
`visits`	one job / service event	Shop software exports job history (preferred)	Customers, vehicles, full visit timeline, lifetime value

Visit imports also serve attribution: re-importing next month's job history lets the system detect who came back (section 14).

Wizard steps

1. **Upload.** CSV, TXT or XLSX, max 20 MB and 100,000 rows. File goes to a private storage bucket at `imports/{company_id}/{import_id}/source.ext`.
2. **Parse.** Server-side in a background job, never in the browser for the final commit:

- Encoding detection: UTF-8, UTF-8 BOM, **Windows-1250** (very common in Czech exports), ISO-8859-2. Show a preview; let the user switch encoding if diacritics look broken.
- Delimiter detection: `;` (default in Czech Excel), `,`, tab.
- Header row detection; skip empty and summary rows.
- XLSX: first sheet by default, sheet picker if several.

3. **Map columns.** Target fields listed below. AI suggests a mapping from headers + 5 sample rows (sample values masked before sending to AI, see section 15). Mappings are saved per company as a template and auto-applied next time.
4. **Declare legal basis.** Either map a consent column or choose one basis for the batch (`consent` / `existing_customer` / `unknown`) and tick the owner confirmation. Stored on the import and on each customer.
5. **Validate.** Row-level results into `import_rows` with status `valid`, `warning`, `invalid`, `duplicate`.
6. **Review.** Summary counts, filter by issue, download issues as CSV, change mapping and re-validate. Rows can be excluded individually.
7. **Commit.** Background job upserts in batches of 500 inside a transaction per batch; the import records created/updated/skipped counts. Lifecycle status is recomputed for affected vehicles.
8. **History.** List of imports with user, file name, type, counts, status, and "Stáhnout chyby". Source files are deleted after 30 days; row staging after 90 days.

Target fields

Field	Required	Normalization
`external_customer_id`	No, strongly preferred	Trimmed string, key for re-imports
`first_name`, `last_name` / `full_name`	One of them	Trim, collapse spaces, title case only if all-caps; split full name on last space
`phone`	Phone or email	E.164 via libphonenumber-js with default region CZ; classify mobile vs fixed line
`email`	Phone or email	Lowercase, syntax check, reject role addresses only as warning
`vehicle_brand`, `vehicle_model`	No	Map brand aliases ("SKODA", "Škoda auto" → "Škoda")

Field	Required	Normalization
`license_plate`	No	Uppercase, remove spaces and dashes for matching, keep display form
`vin`	No	Uppercase, 17 chars, no I/O/Q, else warning
`visit_date` / `last_visit_date`	Visits: yes	Accept `25.9.2026`, `25. 9. 2026`, `2026-09-25`, Excel serial numbers; reject future dates
`service_type` (raw text)	No	Kept raw; mapped to a service category (below)
`job_value` / `last_job_value`	No	Accept `4 700 Kč`, `4700,00`, `4.700,-`; store in haléře
`total_lifetime_value`	No	Used only if no visit history
`marketing_consent`	No	Truthy values: ano, a, yes, 1, true, x
`notes`	No	Stored, never sent to AI, max 2,000 chars

Validation rules

- **Invalid:** no phone and no email; phone not parseable; visit date missing (visit import) or in the future.
- **Warning:** phone is a fixed line (cannot receive SMS); email syntax suspicious; VIN malformed; job value > 200,000 CZK or ⩽ 0; visit older than 10 years.
- **Duplicate within file:** same external ID; or same normalized phone + same last name; or same email.
- **Match against existing data (update, not create):** external ID first, then phone + last name, then email, then VIN/plate for vehicles. Ambiguous matches (two candidates) are flagged for manual review, never auto-merged.
- **Suppression:** rows whose phone or email hash is on the suppression list are imported but marked opted out.

Service category mapping

Raw service types vary by shop ("Výměna oleje + filtr", "SERVIS 60tkm", "přezutí"). Each company has a mapping table from raw text to canonical categories: `regular_service`, `oil_change`, `tyre_change`, `tyre_storage`, `ac_service`, `brakes`, `diagnostics`, `stk`,

`repair_other`. Keyword rules handle most values (e.g. "olej" → oil_change, "pneu", "přezut" → tyre_change, "klima" → ac_service, "brzd" → brakes, "STK", "emise" → stk). AI may suggest mappings for unmatched values; the operator confirms. Unmapped values become `repair_other`.

Integration-ready design

The importer is one implementation of a `DataSource` interface that emits normalized `CustomerRecord` and `VisitRecord` objects. A future garage-software connector implements the same interface and feeds the same staging and upsert pipeline.

12. Segmentation and campaign creation flow

Segments are saved, deterministic rules evaluated in SQL; campaigns snapshot a segment at send time so results stay reproducible.

Lifecycle status (per vehicle, rolled up to customer)

Computed nightly and after each import. Thresholds are company settings with these defaults:

Status	Czech label	Rule (default)
`active`	Aktivní	Last visit ⩽ 9 months ago
`at_risk`	Ohrožený	Last visit 9–12 months ago
`inactive`	Neaktivní	Last visit > 12 months ago
`reactivated`	Vrácený	Completed attributed conversion in the last 12 months (overrides the three above)
`opted_out`	Odhlášený	On suppression list or eligibility withdrawn (overrides everything)

A customer's status is the "best" status of their vehicles (active beats at risk beats inactive), except opted out, which applies to the whole customer.

Marketing eligibility is separate from status: `eligible = legal_basis in (consent, existing_customer) AND not suppressed AND has mobile phone AND last_visit within max_lookback (default 48 months)`.

Preset segments

Preset	Rule
Bez návštěvy 6+ měsíců	last visit (any category) 6–12 months
Bez návštěvy 12+ měsíců	last visit 12–18 months
Bez návštěvy 18+ měsíců	last visit 18–48 months
Pravidelný servis před 10–14 měsíci	last `regular_service` or `oil_change` 10–14 months, no visit since
Pneu minulou sezónu, letos ne	`tyre_change` in same season last year, none this season (spring = Mar–May, autumn = Oct–Nov)
Klimatizace před 18–24 měsíci	last `ac_service` 18–24 months, send Apr–Jun only
Hodnotní zákazníci, kteří zmizeli	lifetime value in top 20% of shop, no visit 9+ months
Pravidelní, kteří přestali chodit	≥ 3 visits in any 24-month window, no visit 12+ months
Blíží se STK	`stk` visit 22–24 months ago (only if the shop recorded STK visits)

Segment rule DSL

Stored as JSON in `segments.rules`, compiled to parameterized SQL by one function (`compileSegment`), unit-tested per operator. No raw SQL from the UI, ever.

```
{
  "all": [
    { "field": "vehicle.last_visit_at", "op": "between_months_ago", "value":
[10, 14], "category": ["regular_service", "oil_change"] },
    { "field": "vehicle.visits_since", "op": "eq", "value": 0 },
    { "field": "customer.lifecycle_status", "op": "in", "value": ["at_risk",
"inactive"] }
  ]
}
```

Operators in MVP: `eq`, `in`, `gte`, `lte`, `between`, `between_months_ago`, `older_than_months`, `season_last_year_not_this`, `percentile_gte`. Groups: `all`, `any`. Eligibility, suppression and frequency caps are always applied after the rule and cannot be turned off.

Contact rules (anti-spam, non-negotiable)

- Max 1 marketing message thread (initial + its follow-ups) per customer per 30 days, across all campaigns.
- Max 4 threads per customer per rolling 12 months.
- Same segment preset not repeated for the same vehicle within 90 days.
- Max 2 follow-ups per campaign; min 2 days between steps.
- Sending window: Mon–Fri 9:00–18:00 and Sat 9:00–12:00, company timezone; never Sundays or Czech public holidays (holiday table seeded for 2026–2030).
- One vehicle per customer per campaign (the most overdue one), so a 3-car customer gets one message.

Campaign builder numbers

Shown value	Formula
Způsobilí zákazníci	Segment matches after eligibility, suppression, caps
Kontrolní skupina	round(eligible × holdout_pct), random, seeded by campaign ID
Odeslat	eligible − holdout
Očekávané tržby	send_count × expected_conversion_rate (default 6%) × avg job value
Maximální potenciál (secondary text)	send_count × avg job value
SMS segmentů / odhad ceny	Σ segments per rendered message × provider price per segment

Average job value: shop's historical median for the segment's service categories if ≥ 20 visits exist, else company `average_job_value`.

Campaign statuses

```
stateDiagram-v2
  [*] --> draft
  draft --> waiting_for_approval: submit (approval required)
  draft --> scheduled: submit (operator confirms)
  waiting_for_approval --> draft: rejected with comment
  waiting_for_approval --> scheduled: approved
  scheduled --> sending: send time reached
  scheduled --> cancelled
  sending --> active: initial step dispatched
  sending --> paused
  sending --> failed: pre-send check or provider failure
  active --> paused
  paused --> active
  paused --> cancelled
  active --> completed: all steps done + attribution window closed
  failed --> draft: operator fixes and resubmits
```

`failed` is added to the brief's status list; without it failures hide inside `paused`. `completed` happens when the attribution window closes, not when the last SMS is sent, so the results are final.

Pre-send checks (run at `scheduled → sending`)

1. Company not paused, global kill switch off, company send switch on.
2. Template renders for every recipient without missing placeholders; opt-out placeholder present.
3. Recipient snapshot re-evaluated: suppression and caps re-checked at send time, not only at creation.
4. Provider credit covers estimated segments × 1.2.
5. Current time within the sending window; otherwise reschedule to next window start.
6. Recipient count within ±20% of the approved count; larger drift pauses and alerts.

13. Automation engine design

Automations are scheduled segment evaluations that produce campaign runs; they are not an event-streaming system. Build them in Phase 4, on top of the same segment, campaign and send pipeline the MVP already uses.

Why schedule-based

"Last service becomes 11 months old" is a time condition, not an event. Evaluating each rule once per day for each company is simple, idempotent and cheap: a shop with 5,000 vehicles is one indexed query. Event triggers (webhooks from garage software) can be added later as a second trigger type that calls the same evaluation.

Model

```
automation_rule
  trigger:   schedule (daily 05:00 company TZ) | seasonal window (e.g. Mar 1–
Apr 15) | [later] event
  segment:   segment_id or inline rules (same DSL as section 12)
  template:  approved message template + follow-up steps
  mode:      require_approval | auto_send
  batching:  weekly (collect new matches Mon–Sun, send Tue 10:00) | daily
  limits:    max recipients per run, max runs per month
  enabled:   bool
```

Run lifecycle

1. Daily job iterates enabled rules per company.
2. Evaluates the segment; subtracts vehicles already enrolled by this rule within its cooldown (default 180 days), plus global contact rules.
3. Adds new matches to the rule's open batch. Idempotency key: `(rule_id, vehicle_id, cooldown_period)`, enforced by a unique index.
4. When the batch window closes, creates a campaign run (a normal `campaign` row with `automation_rule_id` set) with recipient snapshot and holdout.
5. `require_approval` → status `waiting_for_approval` and an alert/notification. `auto_send` → `scheduled`, subject to the same pre-send checks.
6. Results roll up per rule: runs, sent, conversions, revenue.

Batching matters: sending one SMS per day as each car crosses 11 months makes reporting noisy and wastes operator attention. Weekly batches give readable results and one approval per week at most.

Guardrails for auto-send

- Only templates that were sent manually at least once with a delivery rate ⩾ 95% and no complaints can be used with `auto_send`.
- A shop can enable auto-send only after 3 completed campaigns.

- Every auto run obeys the recipient cap; exceeding it switches the run to `require_approval`.
- The global and per-company kill switches stop auto runs immediately.

Example rules (from the brief)

Rule	Trigger	Segment	Default mode
Roční servis	Daily, weekly batch	last regular service 11–13 months, no visit since	require_approval
Přezutí – jaro	Seasonal, Mar 10–Apr 20	tyre change last autumn/spring, none this season	require_approval
Přezutí – podzim	Seasonal, Oct 1–Nov 15	same, autumn	require_approval
Klimatizace	Seasonal, Apr 15–Jun 30	last AC service 18–24 months	require_approval
Neaktivní 12+	Daily, monthly batch	no visit 12–18 months	require_approval
Hodnotní zákazníci	Daily, weekly batch	top 20% LTV, no visit 9+ months	require_approval

MVP preparation: store `automation_rule_id` (nullable) on campaigns and build segment evaluation as a pure function. Do not build the rules UI or scheduler until Phase 4.

14. Revenue attribution design

A completed job counts as recovered revenue when a contacted recipient returns within 60 days of the first message and the job is confirmed; every figure also has a holdout comparison so the shop can see what would have happened anyway.

Definitions

- **Enrollment:** one row per (campaign, customer, vehicle), including holdout rows with `is_holdout = true` and no messages.
- **Attribution window:** default 60 days from the initial message `sent_at` (configurable 30–120 per company). For holdout rows, from the campaign's initial send time.
- **Conversion:** a return visit linked to one enrollment. One conversion per enrollment; if the customer returns twice, the first completed visit counts.

- **Attribution model:** last touch. If a customer is in two campaigns' windows (should be rare given contact caps), the most recent initial message wins.
- **Holdout conversion:** a holdout customer's return in the same window, recorded identically but never counted as recovered revenue.

Conversion pipeline

Stage	Czech label	Set by
`contacted`	Osloven	System when initial SMS delivered (or sent, if no DLR)
`replied`	Odpověděl	System, Phase 4 (two-way SMS only)
`booked`	Objednal se	Manual button, or visit import with a future appointment
`completed`	Dokončeno	Manual with job value, or confirmed import match
`lost`	Neuspěl	System when the window closes without a completed stage
`opted_out`	Odhlásil se	System on opt-out

Three sources of truth, ranked

Source	Confidence	How it works
`import_matched`	High	Visit re-import finds a visit for an enrolled vehicle (plate/VIN) or customer (external ID/phone) within the window. Proposed to operator; confirmed in bulk.
`manual_confirmed`	High	Shop staff or operator marks completed and enters the job value.
`manual_unverified`	Medium	Booked but no job value after 21 days; operator asked to confirm. Not counted until confirmed.

Only completed conversions with a job value count as recovered revenue. Every conversion records who confirmed it and when, and the dashboard can show the split by source.

Formulas

```
\text{Recovered revenue} = \sum_{c \in \text{completed, non-holdout}}
\text{revenue}_c
```

```
\text{ROI} = \frac{\text{Recovered revenue in month}}{\text{Subscription
price in month}}
```

```
\text{Incremental revenue} = \left(r_{\text{treated}} -
r_{\text{holdout}}\right) \times n_{\text{treated}} \times
\overline{\text{job value}}_{\text{treated}}
```

where r is the completed conversion rate within the window. Show incremental revenue only when the campaign's holdout has at least 30 customers; before that show "Nedostatek dat pro porovnání". Across campaigns, pool holdouts per company for a monthly incremental figure.

Other metrics: conversion rate = completed ÷ sent; cost per reactivated customer = (subscription share + SMS cost) ÷ reactivated customers; revenue per segment groups by `campaign.segment_preset`.

Storage

All figures derive from `conversions` and `campaign_enrollments`; nothing is stored as a mutable total. A nightly job writes `company_monthly_stats` (company, month, revenue, conversions, messages, sms_cost, holdout stats) for fast dashboards and is fully rebuildable from raw rows. Editing a conversion marks that month's stats dirty and recomputes it.

15. AI features and exact AI responsibilities

AI writes text a human then approves; deterministic code decides who gets contacted, what data is shown, and what counts as revenue.

Responsibility split

Task	AI	Deterministic code	Human
Choose recipients	Never	Segment rules, eligibility, caps	Picks segment
Write campaign template	Drafts 1 template + 2 alternatives	Validates output	Edits, approves

Task	AI	Deterministic code	Human
Rewrite (kratší / přátelštější / profesionálnější / znovu)	Rewrites	Validates	Chooses
Personalize per recipient	Never	Placeholder rendering	Previews
Czech vocative for surnames	Suggests vocative + gender per unique surname, batched	Caches in `name_vocatives`	Reviews flagged ones
Import column mapping	Suggests from headers + masked samples	Applies mapping, validates	Confirms
Service category mapping	Suggests category for unmapped raw text	Keyword rules first	Confirms
Opportunity cards	Never (MVP)	Rule-based counts and expected revenue	Acts
Revenue attribution	Never	Matching and formulas	Confirms
Replies to customers	Phase 4, draft only	—	Sends

Template placeholders

Placeholder	Rendered from	Fallback
`{{osloveni}}`	"Dobrý den, pane Nováku," if vocative approved and enabled, else "Dobrý den,"	"Dobrý den,"
`{{vozidlo}}`	"vaši Škodu Octavia" (brand + model, accusative handled by lookup table for top 40 brands)	"vaše auto"
`{{posledni_navsteva}}`	"loni v září", "před rokem", "v březnu 2025"	omitted sentence variant
`{{servis}}`	company `default_sender_name`	company name
`{{telefon}}`	company phone, formatted `777 123 456`	required
`{{odhlaseni}}`	"Odhlásit: https://<short>/o/Xk3P9"	required, cannot be removed

The renderer refuses to send a message where a required placeholder is empty. Preview shows 5 random recipients including one with missing vehicle data, so fallbacks are visible before approval.

Generation prompt (skeleton)

```
SYSTEM: You write short SMS messages in natural, polite Czech for an
independent car repair shop.
Rules:
- Use only the placeholders provided. Never invent facts about the customer
or vehicle.
- Never claim the vehicle needs a repair, is unsafe, or is overdue for a
specific part.
- Mention a price or discount only if OFFER is provided; use it verbatim.
- No exclamation marks in more than one sentence. No emoji. No English words.
- Max {max_chars} characters after rendering with typical values.
- Must contain {{servis}} and {{odhlaseni}}. Start with {{osloveni}}.
- Tone: {tone}. Channel: SMS.
Return JSON: {"variants": ["...", "...", "..."]}

USER: Shop: {company_name}, {city}. Campaign goal: {goal_preset_description}.
Service category: {category}. Typical time since last visit: {months_range}.
Offer: {offer_or_none}. Call to action: call {{telefon}}.
```

The AI never receives customer names, phones, emails, plates or notes. It receives only company-level context and the campaign goal. This removes most GDPR exposure from the AI provider.

Output validation (all must pass, else regenerate once, then show error)

- Valid JSON, 1–3 variants, only allowed placeholders.
- Contains `{{servis}}` and `{{odhlaseni}}`.
- Rendered length with typical values ⩽ 320 characters with diacritics (⩽ 5 segments) or ⩽ 459 without (3 segments). Target is 2 segments.
- Denylist regex: "potřebuje", "nutně", "hrozí", "poškozen", "nebezpeč", "zdarma" (unless offer says so), percentages or prices not present in the offer.
- No URLs other than the opt-out placeholder.

Vocatives

Czech vocatives follow patterns (Novák → Nováku, Dvořák → Dvořáku, Svoboda → Svobodo, Černý → Černý, Nováková → Nováková) with many exceptions. Implementation: rule-based first pass, AI batch suggestion for the rest (surname only, no other data), stored per company in `name_vocatives` with `confidence`. Low-confidence entries are listed for operator review. Personalized greeting is off by default per company.

Model and cost

Use a small, fast model through a thin provider wrapper (the Vercel AI SDK works with OpenAI, Anthropic and others). Choose a provider offering EU data processing or zero-data-retention terms and list it as a sub-processor. Generation happens a few times per campaign, so AI costs are negligible (well under 100 CZK per month even at 100 shops). Log every generation in `ai_generations` with prompt version, model, tokens and the chosen variant.

16. SMS and email integration architecture

All outbound messages go through one outbox table and one dispatcher; providers are adapters behind a small interface, so switching provider is a config change plus one new adapter file.

Provider interface

```
export interface MessagingProvider {
  readonly id: string;                          // 'bulkgate' | 'gosms' |
'mock' | ...
  readonly channel: 'sms' | 'email';
  send(msg: OutboundMessage): Promise<SendResult>;          // must accept
idempotencyKey
  parseWebhook(req: Request): Promise<ProviderEvent[]>;     // DLR, inbound,
bounce
  verifyWebhook(req: Request): Promise<boolean>;            // signature /
secret / IP allowlist
  getBalance?(): Promise<{ amount: number; currency: string }>;
  capabilities: { deliveryReports: boolean; inbound: boolean;
alphanumericSender: boolean; unicode: boolean };
}

export type SendResult =
  | { ok: true; providerMessageId: string; segments: number; cost?: {
amountMinor: number; currency: 'CZK' } }
  | { ok: false; retryable: boolean; code: string; detail: string };
```

`getProvider(companyId, channel)` resolves the adapter from `messaging_accounts` (platform default, per-company override later). The dispatcher never imports a concrete provider.

SMS provider selection

Shortlist Czech/EU gateways with REST APIs, delivery reports, alphanumeric sender IDs and CZK invoicing, for example BulkGate, GoSMS.cz or SMSbrána.cz; Twilio as a fallback with higher prices. Verify current pricing, sender ID registration rules for Czech operators, DLR webhooks and inbound support before choosing; this spec does not rely on any provider-specific feature. Always build the `mock` adapter first: it simulates latency, 3% failures and delayed DLRs, and is the only adapter allowed outside production (enforced by an env check at boot).

Message lifecycle

```
stateDiagram-v2
  [*] --> queued
  queued --> sending: dispatcher claims row
  sending --> sent: provider accepted
  sending --> queued: retryable error (backoff)
  sending --> failed: permanent error or 5 retries
  sent --> delivered: DLR
  sent --> undelivered: DLR negative
  sent --> expired: no DLR in 72 h
  queued --> cancelled: opt-out / campaign paused / stop condition
```

Dispatch flow

1. Campaign step due → job renders every message, writes `messages` rows (`queued`) with `idempotency_key = sha256(enrollment_id + step_id)` under a unique index. Re-running the job cannot create duplicates.
2. Dispatcher job claims batches with `SELECT … FOR UPDATE SKIP LOCKED`, respecting provider rate limit (e.g. 10 messages/second, configurable) and the sending window.
3. Before each send: re-check suppression and enrollment stop conditions. Cancelled rows are never sent.
4. On success store `provider_message_id` , `segments` , `cost_minor` , `sent_at` . On retryable error: exponential backoff (1, 5, 15, 60 min). On permanent error: `failed` + reason code.
5. DLR webhooks → `message_events` (raw payload, dedup by provider event ID) → status update. Webhooks return 200 fast and process async.
6. A reconciliation job re-polls status for messages stuck in `sent` for 24 h if the provider supports status lookup.

A message is marked `sending` before the provider call. If the process dies mid-call, rows stuck in `sending` for 10 minutes go to manual review, never to automatic retry: a possible duplicate SMS is worse than a delayed one.

Segment counting and cost

Implement `countSmsSegments(text)` : detect GSM-7 vs UCS-2 (including GSM-7 extension characters that count double, like `€` and `{`), return segments and encoding. Use it in the editor, in estimates and in usage reporting. Store actual segments from the provider when returned, else the computed value. Cost per message comes from the provider response or a price table in `messaging_accounts.price_per_segment_minor` .

Follow-up stop conditions

A follow-up step is cancelled for an enrollment if any is true at render and at dispatch time: customer opted out; enrollment reached `booked`, `completed` or `replied`; a visit was imported after the initial send; the initial message `failed` or `undelivered`; the campaign is paused or cancelled.

Opt-out

- Each message gets a short token (8 chars, random, stored in `opt_out_tokens` with company, customer, message). Link format `https://<short-domain>/o/{token}`. Use a short dedicated domain to save characters.
- The page shows shop name and one button; POST (not GET, so link previews and scanners never unsubscribe anyone) adds the customer to `suppression_list` and cancels queued messages.
- If the provider supports inbound SMS: keywords STOP, ODHLASIT, NE (case- and diacritics-insensitive) do the same.
- Suppression is per company. A customer opting out of Autoservis A is not suppressed for Autoservis B, because B is a different controller.

Email (Phase 3)

Same outbox, `channel = 'email'`. Transactional email (invites, password reset, approval requests, monthly reports) goes through a transactional provider from Phase 1, also used as Supabase Auth SMTP. Marketing email uses an EU-based provider (e.g. Brevo or Mailjet) from its own sending domain per shop with SPF, DKIM and DMARC, one-click `List-Unsubscribe` headers, and bounce/complaint webhooks feeding the suppression list. Open tracking is off by default: Apple Mail privacy protection makes opens unreliable, and tracking pixels raise ePrivacy questions. Click tracking is optional and only via the platform's own redirect domain.

17–19. Multi-tenant architecture and database schema

Use one Postgres database with a shared schema, a `company_id` on every tenant table, Row Level Security on every table, and composite foreign keys so a row can never reference another tenant's row. Schema-per-tenant or database-per-tenant is unnecessary below several thousand shops and makes migrations painful.

17. Tenant isolation, four layers

1. **RLS on every table** in `public`, enabled and forced. No table ships without policies; a CI check fails the build if `pg_tables` shows a table with RLS disabled.

2. **Composite foreign keys.** Parent tables have `unique (id, company_id)`; children reference `(parent_id, company_id)`. A vehicle cannot point to a customer of another company even through a bug in service-role code.
3. **Service-role code is scoped.** Background jobs use the service role (RLS bypassed). Every job receives `companyId` explicitly and uses a `db.forCompany(companyId)` helper that adds the filter; raw service-role queries without it are banned by lint rule and code review.
4. **Automated cross-tenant tests.** For every table: user of company A tries select/insert/update/delete on company B's rows and must get zero rows or an error (section 30).

18. Proposed schema

Conventions: `uuid` primary keys (`gen_random_uuid()`), `timestamptz` for instants, `date` for visit dates, money as `bigint` in haléře (`*_minor`), English snake_case, `created_at` / `updated_at` on every table, enums as Postgres enums (add values via migration).

```
-- ============ ENUMS ============
create type company_role      as enum ('shop_owner','shop_manager');
-- extend later
create type service_mode      as enum ('managed','self_service');
create type legal_basis       as enum
('consent','existing_customer','unknown');
create type lifecycle_status  as enum
('active','at_risk','inactive','reactivated','opted_out');
create type service_category  as enum
('regular_service','oil_change','tyre_change','tyre_storage',

'ac_service','brakes','diagnostics','stk','repair_other');
create type campaign_status   as enum
('draft','waiting_for_approval','scheduled','sending','active',

'paused','completed','cancelled','failed');
create type message_status    as enum
('queued','sending','sent','delivered','undelivered','expired',
                                       'failed','cancelled');
create type conversion_stage  as enum
('contacted','replied','booked','completed','lost','opted_out');
create type conversion_source as enum
('manual_confirmed','import_matched','manual_unverified');

-- ============ PLATFORM & TENANCY ============
create table platform_admins (
```

```
  user_id uuid primary key references auth.users on delete cascade,
  created_at timestamptz not null default now()
);

create table companies (
  id uuid primary key default gen_random_uuid(),
  name text not null, ico text, dic text,
  address_line text, city text, postal_code text, country text not null
default 'CZ',
  phone text, email text, website text, logo_path text, contact_person text,
  default_locale text not null default 'cs', timezone text not null default
'Europe/Prague',
  average_job_value_minor bigint not null default 350000,
  default_sender_name text check (char_length(default_sender_name) <= 11),
  preferred_channel text not null default 'sms',
  opening_hours jsonb not null default '{}',
  service_mode service_mode not null default 'managed',
  requires_client_approval boolean not null default false,
  settings jsonb not null default '{}',   -- lifecycle thresholds, lookback,
attribution window, holdout pct, margin
  sending_enabled boolean not null default false,
  dpa_signed_at timestamptz, dpa_document_path text,
  status text not null default 'onboarding'
    check (status in
('onboarding','active','paused','offboarding','closed')),
  created_at timestamptz not null default now(), updated_at timestamptz not
null default now()
);

create table profiles (
  user_id uuid primary key references auth.users on delete cascade,
  full_name text, locale text not null default 'cs',
  created_at timestamptz not null default now(), updated_at timestamptz not
null default now()
);

create table memberships (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references auth.users on delete cascade,
  company_id uuid not null references companies on delete cascade,
  role company_role not null,
  created_at timestamptz not null default now(),
  unique (user_id, company_id)
);

create table invitations (
```

```
  id uuid primary key default gen_random_uuid(),
  company_id uuid not null references companies on delete cascade,
  email text not null, role company_role not null,
  token_hash text not null unique, expires_at timestamptz not null,
accepted_at timestamptz,
  invited_by uuid references auth.users, created_at timestamptz not null
default now()
);

-- ============ CUSTOMER DATA ============
create table customers (
  id uuid primary key default gen_random_uuid(),
  company_id uuid not null references companies on delete cascade,
  external_id text,
  first_name text, last_name text,
  phone_e164 text, phone_type text check (phone_type in
('mobile','fixed','unknown')),
  email text,
  legal_basis legal_basis not null default 'unknown',
  legal_basis_source text, legal_basis_recorded_at timestamptz,
  lifecycle_status lifecycle_status not null default 'inactive',
  last_visit_at date, visit_count int not null default 0,
  lifetime_value_minor bigint not null default 0,
  notes text check (char_length(notes) <= 2000),
  source_import_id uuid,
  created_at timestamptz not null default now(), updated_at timestamptz not
null default now(),
  unique (id, company_id)
);
create unique index on customers (company_id, external_id) where external_id
is not null;
create index on customers (company_id, phone_e164);
create index on customers (company_id, lifecycle_status, last_visit_at);

create table vehicles (
  id uuid primary key default gen_random_uuid(),
  company_id uuid not null,
  customer_id uuid not null,
  brand text, model text, model_year int,
  license_plate_display text, license_plate_norm text, vin text,
  last_visit_at date, lifecycle_status lifecycle_status not null default
'inactive',
  created_at timestamptz not null default now(), updated_at timestamptz not
null default now(),
  unique (id, company_id),
  foreign key (customer_id, company_id) references customers (id, company_id)
```

```
on delete cascade
);
create unique index on vehicles (company_id, vin) where vin is not null;
create index on vehicles (company_id, license_plate_norm);

create table visits (
  id uuid primary key default gen_random_uuid(),
  company_id uuid not null,
  customer_id uuid not null,
  vehicle_id uuid,
  visited_at date not null,
  service_type_raw text, service_category service_category not null default
'repair_other',
  job_value_minor bigint, external_ref text, source_import_id uuid,
  created_at timestamptz not null default now(),
  unique (id, company_id),
  foreign key (customer_id, company_id) references customers (id, company_id)
on delete cascade,
  foreign key (vehicle_id, company_id) references vehicles (id, company_id)
on delete set null (vehicle_id)
);
create unique index on visits (company_id, external_ref) where external_ref
is not null;
create index on visits (company_id, vehicle_id, service_category, visited_at
desc);

create table service_category_mappings (
  id uuid primary key default gen_random_uuid(),
  company_id uuid not null references companies on delete cascade,
  raw_value_norm text not null, category service_category not null,
  created_by uuid references auth.users, created_at timestamptz not null
default now(),
  unique (company_id, raw_value_norm)
);

create table name_vocatives (
  id uuid primary key default gen_random_uuid(),
  company_id uuid not null references companies on delete cascade,
  surname text not null, vocative text not null,
  gender text check (gender in ('m','f','unknown')), confidence numeric(3,2)
not null,
  reviewed_by uuid references auth.users, reviewed_at timestamptz,
  unique (company_id, surname)
);

-- ============ IMPORTS ============
```

```
create table imports (
  id uuid primary key default gen_random_uuid(),
  company_id uuid not null references companies on delete cascade,
  type text not null check (type in ('customers','visits')),
  file_path text, file_name text, file_size_bytes int, encoding text,
delimiter text,
  mapping jsonb, legal_basis_declared legal_basis,
  legal_basis_confirmed_by uuid references auth.users,
legal_basis_confirmed_at timestamptz,
  status text not null default 'uploaded' check (status in
('uploaded','parsing','mapping','validating',
    'needs_review','committing','completed','failed','cancelled')),
  counts jsonb not null default '{}', error text,
  created_by uuid references auth.users, created_at timestamptz not null
default now(),
  completed_at timestamptz,
  unique (id, company_id)
);

create table import_rows (
  id bigint generated always as identity primary key,
  import_id uuid not null, company_id uuid not null,
  row_number int not null, raw jsonb not null, normalized jsonb,
  status text not null check (status in
('valid','warning','invalid','duplicate','excluded')),
  issues jsonb not null default '[]', match_customer_id uuid,
  foreign key (import_id, company_id) references imports (id, company_id) on
delete cascade
);
create index on import_rows (import_id, status);

create table import_mapping_templates (
  company_id uuid not null references companies on delete cascade,
  type text not null, mapping jsonb not null, updated_at timestamptz not null
default now(),
  primary key (company_id, type)
);

-- ============ SEGMENTS & CAMPAIGNS ============
create table segments (
  id uuid primary key default gen_random_uuid(),
  company_id uuid not null references companies on delete cascade,
  name text not null, preset_key text, rules jsonb not null,
  created_by uuid references auth.users, archived_at timestamptz,
  created_at timestamptz not null default now(), updated_at timestamptz not
null default now(),
```

```
  unique (id, company_id)
);

create table campaigns (
  id uuid primary key default gen_random_uuid(),
  company_id uuid not null,
  name text not null, segment_id uuid, segment_preset text,
segment_rules_snapshot jsonb,
  channel text not null default 'sms', status campaign_status not null
default 'draft',
  scheduled_at timestamptz, holdout_pct numeric(4,3) not null default 0.10,
  offer_text text, automation_rule_id uuid,
  expected_conversion_rate numeric(4,3) not null default 0.06,
  avg_job_value_minor_snapshot bigint, attribution_window_days int not null
default 60,
  approved_by uuid references auth.users, approved_at timestamptz,
approval_comment text,
  created_by uuid references auth.users,
  created_at timestamptz not null default now(), updated_at timestamptz not
null default now(),
  completed_at timestamptz,
  unique (id, company_id),
  foreign key (segment_id, company_id) references segments (id, company_id)
);
create index on campaigns (company_id, status);

create table campaign_steps (
  id uuid primary key default gen_random_uuid(),
  company_id uuid not null, campaign_id uuid not null,
  step_number int not null check (step_number between 0 and 2),
  delay_days int not null default 0, body_template text not null,
  strip_diacritics boolean not null default false,
  created_at timestamptz not null default now(),
  unique (id, company_id), unique (campaign_id, step_number),
  foreign key (campaign_id, company_id) references campaigns (id, company_id)
on delete cascade
);

create table campaign_enrollments (
  id uuid primary key default gen_random_uuid(),
  company_id uuid not null, campaign_id uuid not null,
  customer_id uuid not null, vehicle_id uuid,
  is_holdout boolean not null default false,
  stage conversion_stage not null default 'contacted',
  stopped_reason text, first_sent_at timestamptz, window_ends_at timestamptz,
  created_at timestamptz not null default now(), updated_at timestamptz not
```

```
null default now(),
  unique (id, company_id), unique (campaign_id, customer_id),
  foreign key (campaign_id, company_id) references campaigns (id, company_id)
on delete cascade,
  foreign key (customer_id, company_id) references customers (id, company_id)
on delete cascade
);
create index on campaign_enrollments (company_id, customer_id, created_at
desc);

-- ============ MESSAGING ============
create table messaging_accounts (
  id uuid primary key default gen_random_uuid(),
  company_id uuid references companies on delete cascade,   -- null =
platform default
  provider_id text not null, channel text not null, sender_id text,
  config jsonb not null default '{}',                       -- non-secret
only
  price_per_segment_minor int, is_active boolean not null default true
);

create table messages (
  id uuid primary key default gen_random_uuid(),
  company_id uuid not null, campaign_id uuid, enrollment_id uuid, step_id
uuid, customer_id uuid,
  channel text not null default 'sms', to_address text not null,
  body_rendered text not null, encoding text, segments int,
  status message_status not null default 'queued',
  provider_id text, provider_message_id text,
  idempotency_key text not null unique,
  cost_minor int, error_code text, error_detail text, attempts int not null
default 0,
  next_attempt_at timestamptz, queued_at timestamptz not null default now(),
  sent_at timestamptz, delivered_at timestamptz, failed_at timestamptz,
  foreign key (enrollment_id, company_id) references campaign_enrollments
(id, company_id) on delete set null (enrollment_id)
);
create index on messages (status, next_attempt_at) where status in
('queued','sending');
create index on messages (company_id, sent_at);
create index on messages (provider_id, provider_message_id);

create table message_events (
  id bigint generated always as identity primary key,
  company_id uuid, message_id uuid, provider_id text not null,
provider_event_id text,
```

```
  type text not null, payload jsonb not null, received_at timestamptz not
null default now(),
  unique (provider_id, provider_event_id)
);

create table opt_out_tokens (
  token text primary key,                                    -- 8 random
base62 chars
  company_id uuid not null, customer_id uuid, message_id uuid,
  created_at timestamptz not null default now(), used_at timestamptz
);

-- ============ CONVERSIONS & STATS ============
create table conversions (
  id uuid primary key default gen_random_uuid(),
  company_id uuid not null, enrollment_id uuid not null unique,
  campaign_id uuid not null, customer_id uuid not null, vehicle_id uuid,
  stage conversion_stage not null, source conversion_source not null,
  is_holdout boolean not null default false,
  booked_at timestamptz, completed_at timestamptz, revenue_minor bigint check
(revenue_minor >= 0),
  visit_id uuid, confirmed_by uuid references auth.users, confirmed_at
timestamptz, notes text,
  created_at timestamptz not null default now(), updated_at timestamptz not
null default now(),
  foreign key (enrollment_id, company_id) references campaign_enrollments
(id, company_id) on delete cascade
);
create index on conversions (company_id, stage, completed_at);

create table company_monthly_stats (
  company_id uuid not null references companies on delete cascade, month date
not null,
  recovered_revenue_minor bigint not null default 0, completed_count int not
null default 0,
  booked_count int not null default 0, messages_sent int not null default 0,
  sms_segments int not null default 0, sms_cost_minor bigint not null default
0,
  holdout_stats jsonb not null default '{}', computed_at timestamptz not null
default now(),
  primary key (company_id, month)
);

-- ============ COMPLIANCE & OPERATIONS ============
create table suppression_list (
  id uuid primary key default gen_random_uuid(),
```

```
  company_id uuid not null references companies on delete cascade,
  channel text not null, address_hash bytea not null,       -- HMAC-
SHA256(normalized address, SUPPRESSION_PEPPER)
  reason text not null check (reason in
('opt_out_link','keyword','manual','erasure','bounce','complaint')),
  created_at timestamptz not null default now(),
  unique (company_id, channel, address_hash)
);

create table data_subject_requests (
  id uuid primary key default gen_random_uuid(),
  company_id uuid not null references companies on delete cascade,
  type text not null check (type in
('access','erasure','rectification','objection')),
  subject_reference text not null, status text not null default 'open',
  due_at date not null, completed_at timestamptz, handled_by uuid references
auth.users,
  created_at timestamptz not null default now()
);

create table audit_logs (
  id bigint generated always as identity primary key,
  company_id uuid, actor_user_id uuid, actor_type text not null check
(actor_type in ('user','admin','system')),
  action text not null, entity_type text, entity_id text, metadata jsonb not
null default '{}',
  ip inet, user_agent text, created_at timestamptz not null default now()
);
create index on audit_logs (company_id, created_at desc);

create table admin_access_sessions (
  id uuid primary key default gen_random_uuid(),
  admin_user_id uuid not null references auth.users, company_id uuid not null
references companies,
  reason text not null, mode text not null check (mode in ('read','write')),
  started_at timestamptz not null default now(), ended_at timestamptz
);

create table alerts (
  id uuid primary key default gen_random_uuid(),
  company_id uuid references companies on delete cascade,
  type text not null, severity text not null check (severity in
('critical','high','medium','low','info')),
  dedupe_key text not null, title text not null, detail jsonb not null
default '{}',
  status text not null default 'open' check (status in
```

```
('open','acknowledged','resolved')),
  acknowledged_by uuid, acknowledged_at timestamptz, resolved_at timestamptz,
  created_at timestamptz not null default now()
);
create unique index on alerts (dedupe_key) where status <> 'resolved';

create table notifications (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references auth.users on delete cascade, company_id
uuid,
  type text not null, payload jsonb not null default '{}', read_at
timestamptz,
  created_at timestamptz not null default now()
);

create table ai_generations (
  id uuid primary key default gen_random_uuid(),
  company_id uuid not null references companies on delete cascade,
campaign_id uuid,
  kind text not null, prompt_version text not null, model text not null,
  input jsonb not null, output jsonb, chosen_index int, tokens_in int,
tokens_out int,
  created_by uuid references auth.users, created_at timestamptz not null
default now()
);

create table public_holidays (day date primary key, country text not null
default 'CZ', name text not null);

create table feature_flags (
  key text not null, company_id uuid references companies on delete cascade,
  enabled boolean not null default false
);
create unique index on feature_flags (key, company_id) nulls not distinct;
-- null company_id = global flag

-- ============ BILLING ============
create table plans (
  id uuid primary key default gen_random_uuid(),
  key text not null unique, name text not null,
  price_minor bigint not null, currency text not null default 'CZK',
  limits jsonb not null default '{}',   -- {"customers": 5000,
"sms_segments_per_month": 800}
  is_active boolean not null default true
);
```

```
create table subscriptions (
  id uuid primary key default gen_random_uuid(),
  company_id uuid not null unique references companies on delete cascade,
  plan_id uuid not null references plans, price_override_minor bigint,
  status text not null check (status in
('trial','active','past_due','cancelled')),
  trial_ends_at date, current_period_start date, current_period_end date,
  billing_provider text not null default 'manual', external_ref text,
  created_at timestamptz not null default now(), updated_at timestamptz not
null default now()
);
```

Phase 4 tables (do not create earlier): `automation_rules`, `automation_enrollments` (unique `(rule_id, vehicle_id, cooldown_period)`), `inbound_messages`.

RLS pattern

```
create schema if not exists app;

create function app.is_platform_admin() returns boolean
  language sql stable security definer set search_path = '' as $$
  select exists (select 1 from public.platform_admins where user_id = (select
auth.uid()));
$$;

create function app.company_role(cid uuid) returns public.company_role
  language sql stable security definer set search_path = '' as $$
  select role from public.memberships where company_id = cid and user_id =
(select auth.uid());
$$;

alter table public.customers enable row level security;
alter table public.customers force row level security;

create policy customers_select on public.customers for select to
authenticated
  using (app.company_role(company_id) is not null or
app.is_platform_admin());

create policy customers_update on public.customers for update to
authenticated
  using (app.company_role(company_id) in ('shop_owner','shop_manager') or
app.is_platform_admin())
  with check (app.company_role(company_id) in ('shop_owner','shop_manager')
or app.is_platform_admin());
```

```
-- customers insert/delete: platform admin only in MVP (imports run in jobs)
create policy customers_delete on public.customers for delete to
authenticated
  using (app.is_platform_admin());
```

Apply the same pattern to every tenant table. Tables with no client access (`messages` writes, `message_events` , `opt_out_tokens` , `import_rows` , `ai_generations` , `alerts` , `audit_logs`) get select-only or no policies for `authenticated` ; jobs write via the service role. `audit_logs` has no update or delete policy for anyone, and a trigger rejects updates and deletes.

19. Important fields explained

Field	Why it matters
`customers.legal_basis`	Legal gate for sending; `unknown` never receives marketing
`companies.sending_enabled`	Off until DPA is signed and the first import is reviewed
`companies.settings`	All per-shop thresholds in one validated JSON (Zod schema), with defaults in code
`campaigns.segment_rules_snapshot`	Reproducible results even if the segment is edited later
`campaign_enrollments.is_holdout`	Makes incremental revenue measurable
`messages.idempotency_key`	Prevents double sends on retries and job re-runs
`suppression_list.address_hash`	Keeps opt-outs after customer data is erased, without storing the phone
`conversions.source`	Shows how trustworthy each revenue figure is
`alerts.dedupe_key`	One alert per issue, not one per health-check run

20. Authentication and authorization model

Supabase Auth handles identity; authorization is a permission check in the app plus RLS in the database, and both must agree. Access is invite-only until Phase 3.

Authentication

- Email + password (min. 12 characters, checked against leaked-password lists via Supabase's setting), password reset by email link, email verification on invite acceptance.
- **Super admins must use TOTP MFA.** Admin routes check `aal2` in the session; without it, redirect to MFA enrolment.
- Sessions via Supabase SSR cookies (`@supabase/ssr`), HTTP-only, `SameSite=Lax`, refresh handled by middleware.
- Sign-up: disabled in Supabase settings. Users join only through `invitations` (token hashed in DB, 7-day expiry, single use).
- Rate limits: 5 failed logins per 15 minutes per email + IP, then a 15-minute lockout; generic error messages that don't reveal whether an email exists.

Roles

Platform role (`platform_admins`) is separate from company roles (`memberships.role`). This keeps super admin out of the tenant model and makes future roles (e.g. `platform_operator` with limited rights, `shop_viewer`) additive.

Permission matrix

Permissions live in one TypeScript map (`lib/auth/permissions.ts`). UI hides actions the user lacks; server actions re-check with `assertCan(user, permission, companyId)`; RLS is the final guard.

Permission	super_admin	shop_owner (managed)	shop_owner (self-service)	shop_manager
`dashboard.view`	✓	✓	✓	✓
`customers.view`	✓	✓	✓	✓
`customers.edit_contact`	✓	✓	✓	✓
`customers.export`	✓	✓	✓	—
`customers.delete`	✓	request only	✓	—
`conversions.mark`	✓	✓	✓	✓
`imports.run`	✓	—	✓	—
`segments.manage`	✓	—	✓	—
`campaigns.create`	✓	—	✓	—
`campaigns.approve`	✓	✓ if `requires_client_approval`	✓	—
`campaigns.pause`	✓	✓	✓	—
`settings.company`	✓	✓	✓	—

Permission	super_admin	shop_owner (managed)	shop_owner (self-service)	shop_manager
`users.invite`	✓	✓	✓	—
`billing.view`	✓	✓	✓	—
`admin.*` (all shops, alerts, system)	✓	—	—	—

The owner can always pause a campaign, even in managed mode. It is their brand and their customers.

Super admin access to a shop

- The admin app reads any company through RLS (`app.is_platform_admin()`), but tenant pages under `/admin/companies/[id]/…` log `admin.company_viewed` once per session per company.
- "Otevřít účet klienta" creates an `admin_access_sessions` row with a reason and renders the client UI in read-only mode with the banner.
- Write mode requires re-confirming MFA within the last 15 minutes and is logged separately. All writes carry `actor_type = 'admin'` in the audit log.
- No token swapping and no "log in as". The admin never holds a session for the client's user account.

Server-side rules

- The Supabase service role key exists only in server runtime and job workers, never in client bundles (checked by a build step that greps the output for the key prefix).
- Every server action and route handler starts with `const ctx = await requireContext({ permission, companyId })` , which resolves the user, their membership and the admin flag once.
- `companyId` in URLs is always verified against membership; never trust it on its own.

21–23. GDPR, security architecture, audit logging

The shop is the controller, you are the processor, and the product must make the compliant path the default path. This section is an engineering checklist; confirm the legal parts with a Czech data-protection lawyer before Phase 2.

21. GDPR and privacy

Roles and paperwork (before the first real import)

- Processing agreement (Art. 28) with each shop: subject matter, duration, data categories, instructions, sub-processor list, breach notice within 24 h to the shop, deletion at contract end.
- Your own records of processing (Art. 30) as a processor.
- Sub-processor list published on your website: hosting/database, job runner, SMS provider, AI provider, email provider, error monitoring. Prefer EU regions for all; where a US provider is used, rely on the EU–US Data Privacy Framework or SCCs and note it.
- A short privacy notice template the shop can place at its reception and website, covering reminders by SMS.
- A lightweight DPIA note. Large-scale profiling is not happening here, but a written assessment is cheap and useful if ÚOOÚ asks.

Data minimization

Data	Stored	Reason
Name, phone, email	Yes	Contact and greeting
Vehicle brand/model/plate/VIN	Yes	Service intervals, matching returns
Visit dates, categories, job values	Yes	Segments and attribution
Free-text notes	Optional, max 2,000 chars, never sent to AI	Shops often need context; warn not to store health or sensitive data
Birth date, address of customer, national ID	No	Not needed; importer ignores these columns
Payment data of end customers	No	Not needed

Data subject rights

- Access: admin action exports one customer's data as JSON + PDF summary within the 30-day deadline.
- Erasure: hard-deletes customer, vehicles, visits, enrollments and message bodies; keeps the suppression hash (legitimate interest in honouring the objection) and anonymized revenue totals in stats.
- Objection to direct marketing (Art. 21): absolute. Opt-out link, keyword, email or phone request all end up in the suppression list.
- Requests are tracked in `data_subject_requests` with due dates and an alert at day 20.

Retention defaults

Data	Retention
Uploaded import files	30 days, then deleted from storage
`import_rows` staging	90 days
Message bodies	24 months, then body replaced with `[redacted]`, metadata kept
Customers with no visit in 5 years and no contact in 2	Flagged for deletion; deleted after 30 days unless shop objects
Audit logs	24 months
Everything of an offboarded shop	Export offered, then deleted within 30 days of contract end; backups age out within 35 days

Retention runs as a nightly job with its own audit entries.

22. Security architecture

Data protection

- Supabase region EU (Frankfurt); Vercel functions pinned to `fra1`; job runner in EU where available.
- TLS everywhere; encryption at rest by the platform (AES-256). Column-level encryption of phones is not worth the complexity in MVP because the app must read them to send SMS; RLS + disk encryption + access control is the right level.
- Suppression list stores only HMAC hashes with a server-side pepper.
- Storage buckets private; paths prefixed by `company_id`; storage RLS policies mirror table policies; downloads only via 60-second signed URLs.

Secure CSV processing

- Size (20 MB) and row (100,000) limits enforced before parsing; MIME sniffing plus extension check; reject macro-enabled `.xlsm` and zip bombs (XLSX uncompressed size limit 100 MB).
- Parsing happens in the job worker, never with `eval` or formula evaluation; cell values are treated as strings.
- **CSV injection on export:** when exporting CSV, prefix cells starting with `=`, `+`, `-`, `@`, tab or CR with a single quote.
- Raw rows are stored only in `import_rows` (tenant-scoped, 90-day retention).

Application security

- All inputs validated with Zod on the server; output encoded by React; no `dangerouslySetInnerHTML`.
- Security headers: CSP (self + required providers), HSTS, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy` minimal.
- CSRF: server actions include origin checks by default; route handlers that mutate check `Origin`.
- Rate limits (Upstash Redis or Vercel's rate limiting): login, password reset, invitation accept, AI generation (30/hour/user), opt-out endpoint (per IP), webhooks (per provider).
- Webhooks: verify signature or shared secret in header, plus provider IP allowlist where published; reject stale timestamps; store raw payload; process idempotently.
- Global and per-company **send kill switch** in the admin System page, checked by the dispatcher before every batch.

Secrets management

- Secrets only in Vercel and job runner environment settings; never in the repo. `.env.example` has names only.
- Separate keys per environment; production keys are never on a developer laptop.
- Rotate SMS/AI/service-role keys at least yearly and immediately after any team change.
- GitHub: secret scanning and push protection on, Dependabot on, branch protection on `main`.

Development data rule: no real customer data outside production. Local and staging use seed data only. There is no "copy production to staging" script, and the importer in non-production refuses files whose phone numbers look real unless a `DEV_ALLOW_REAL_LOOKING_DATA` flag is set for a specific test.

23. Audit logging

One append-only `audit_logs` table, written through one function `audit(ctx, action, entity, metadata)`. Metadata holds IDs and changed field names, never full personal data (no phone numbers in logs).

Action	Actor	Metadata
`auth.login`, `auth.login_failed`, `auth.mfa_enrolled`	user	IP, user agent

Action	Actor	Metadata
`user.invited`, `user.role_changed`, `user.removed`	user/admin	target user, role
`company.created`, `company.settings_changed`	admin/user	changed keys
`company.dpa_confirmed`, `company.sending_enabled`	admin	document path
`import.uploaded`, `import.committed`, `import.failed`	user/system	counts, legal basis
`customer.updated`, `customer.deleted`, `customer.exported`	user/admin	fields changed, count
`customer.opted_out`	system/user	channel, source
`campaign.created`, `.submitted`, `.approved`, `.rejected`, `.scheduled`, `.paused`, `.cancelled`	user/admin	recipient count
`campaign.send_started`, `campaign.send_completed`, `campaign.failed`	system	counts, errors
`conversion.marked`, `conversion.revenue_changed`, `conversion.confirmed_bulk`	user/admin	old/new value
`admin.company_viewed`, `admin.support_view_started`, `admin.write_mode_enabled`	admin	reason
`dsr.created`, `dsr.completed`	admin	type
`system.kill_switch_changed`, `system.retention_run`	admin/system	scope, counts

Audit log UI: per company tab in admin, filter by action and date; shop owners see a simplified "Historie změn" in Settings for their own company.

24–25. Error handling and monitoring

Errors that touch sending fail closed (stop, alert, wait for a human); errors elsewhere fail visibly with a Czech message the user can act on.

24. Error handling

Error classes (in `lib/errors.ts`): `ValidationError` (400, show field messages), `AuthError` (401/403, redirect or "Nemáte oprávnění"), `NotFoundError` (404), `ConflictError` (409, e.g. campaign already sending), `ProviderError` (retryable flag, provider code), `InternalError` (500, generic message + error ID).

Server actions return a discriminated result `{ ok: true, data } | { ok: false, error: { code, message, fields? } }` instead of throwing to the client. Messages are i18n keys.

Jobs

Job	On transient error	On permanent error
Import parse/commit	Retry 3× with backoff; commit is per-batch transactional and resumable from last batch	Import → `failed`, error CSV, alert
Campaign render	Retry 3×	Campaign → `failed`, alert critical
Dispatcher	Per-message backoff 1/5/15/60 min	Message `failed` with code; campaign failure alert if > 5%
Stuck in `sending` > 10 min	No automatic retry	Manual review queue, alert high
DLR webhook processing	Retry via job runner	Store raw event, alert if parse fails > 10 in 1 h
Nightly lifecycle/stats	Retry 3×	Alert high; dashboards show "Data k 24. 9." with the last good timestamp
AI generation	Retry once	Show "Generování se nepovedlo, zkuste to znovu nebo napište zprávu ručně"

Circuit breaker on the SMS provider: if more than 20% of sends in the last 5 minutes fail with provider errors, the dispatcher pauses the whole provider for 15 minutes and raises a critical alert. Campaigns resume automatically after the pause if the next probe succeeds; otherwise they stay paused.

UI: every page has an error boundary with retry and error ID; toast notifications for action results; forms keep input on failure.

25. Monitoring

Concern	Tool	Setup
Exceptions (web + jobs)	Sentry (EU data region)	Source maps, release tags, PII scrubbing on (no phone/email in events), alerts to email
Uptime	Better Stack or UptimeRobot	`/api/health` (DB ping, provider credit check cached) every 1 min; public opt-out page monitored separately
Jobs	Inngest dashboard	Failure alerts to email/Slack; function-level concurrency limits visible
Database	Supabase dashboard + advisors	Weekly check of security and performance advisors; slow query log
Product analytics	PostHog (EU cloud), Phase 2	Only operator and shop-user events; no end-customer data; autocapture off
Business health	The app's own Attention Required	Section 10 rules
Logs	Vercel logs + structured JSON logger (pino)	Request ID and company ID on every log line; no PII

Daily 06:00 digest email to the operator: alerts opened/resolved in the last 24 h, messages sent, failures, campaigns scheduled today. If everything is fine, the subject line says so: "Vše v pořádku · 3 kampaně dnes".

26–28. Technology stack, third-party services, operating costs

Your proposed stack is right in its core (Next.js, TypeScript, Supabase Postgres, Vercel, Tailwind, shadcn/ui); it needs a durable job runner added, Stripe deferred, and the AI provider kept swappable.

26. Recommended stack

Layer	Choice	vs. your idea	Reason
Framework	Next.js (App Router), TypeScript strict	Keep	Best-supported by AI coding agents; one codebase for UI and API
UI	Tailwind CSS + shadcn/ui, Recharts (via shadcn charts), TanStack Table, react-hook-form + Zod	Keep, add libs	Standard, well known to agents
Database	Supabase Postgres, EU (Frankfurt) region	Keep	Postgres + RLS + auth + storage in one managed service
Data access	supabase-js with the user's session for tenant requests (RLS enforced); typed via `supabase gen types`	Clarify	ORMs like Prisma connect as a privileged role and bypass RLS unless carefully configured; skip them
Migrations	Supabase CLI SQL migrations in the repo	Add	Reviewable, reproducible, agents write SQL well
Auth	Supabase Auth with TOTP MFA	Keep	Built in, RLS-aware
File storage	Supabase Storage, private buckets	Add	Same RLS model
Background jobs	**Inngest** (alternative: Trigger.dev)	**Add, required**	Durable steps, `sleep` for follow-ups, retries, concurrency limits, cron; works with Vercel
Hosting	Vercel Pro, region `fra1`	Keep	Hobby plan does not allow commercial use
AI	Vercel AI SDK + one provider (OpenAI or Anthropic), small model	Change: abstract it	Provider becomes a config value; pick on EU processing terms and Czech quality
SMS	Czech gateway behind adapter	Keep	See section 16

Layer	Choice	vs. your idea	Reason
Transactional email	Resend or Postmark (or Brevo for EU hosting)	Add	Invites, resets, approvals, monthly report
Billing	Fakturoid (manual → API) in Phases 1–2; Stripe Billing + Fakturoid in Phase 3	Change	Czech invoices and bank transfers first; card payments when self-service exists
Rate limiting	Upstash Redis (EU)	Add	Serverless-friendly
Errors	Sentry	Add	Standard
Uptime	Better Stack	Add	Cheap, status page included
Analytics	PostHog EU, Phase 2	Add later	Product usage, not end-customer tracking
i18n	next-intl, messages in `messages/cs.json`	Add	Czech first, other locales later
Parsing	Papa Parse (CSV), SheetJS (XLSX, installed from the official SheetJS CDN tarball, since the npm registry copy is outdated), iconv-lite (Windows-1250), libphonenumber-js, date-fns	Add	Proven
Testing	Vitest, Playwright, pgTAP or SQL tests via Supabase CLI	Add	Section 30
Fake data	@faker-js/faker with `cs_CZ` locale	Add	Realistic Czech names and addresses

Rejected on purpose: microservices, Kubernetes, separate backend (NestJS/Go), GraphQL, Redis queues you host yourself, a data warehouse, Kafka, a vector database. None of them helps prove that customers come back.

27. Third-party services and sub-processors

Service	Purpose	Personal data?	Region to choose	Phase
Supabase	DB, auth, storage	Yes	EU (Frankfurt)	0
Vercel	Hosting, functions	Yes, in transit	`fra1` functions	0
Inngest	Jobs	Job payloads: pass IDs only, not personal data	EU if offered	1
SMS gateway	Sending	Phone + message	CZ/EU	1 (mock in 0)
AI provider	Templates, mapping hints	No personal data by design	EU processing / ZDR	1
Transactional email	System email	Staff emails	EU preferred	1
Sentry	Errors	Scrubbed	EU	1
Upstash	Rate limits	IP addresses	EU	1
Fakturoid	Invoicing shops	Shop business data	CZ	2
Better Stack	Uptime	No	EU	2
PostHog	Product analytics	Staff usage	EU	2
Stripe	Card payments	Shop billing	EU entity	3
Brevo/Mailjet	Marketing email	Emails	EU	3

28. Estimated operating costs

Approximate monthly figures, based on list prices remembered as of mid-2026 at about 22 CZK per USD; verify current pricing before relying on them. Staff costs are excluded.

Per-shop SMS assumption: 250 recipients per month, 1.4 messages each with follow-ups, 1.5 segments per message on average, about 1 CZK per segment ≈ **525 CZK per shop per month**. SMS is the dominant variable cost, not infrastructure.

Clients	Infra and SaaS	SMS	Total cost	MRR at 4,990 Kč	Gross margin
5	~1 100 Kč (Vercel Pro, Supabase Pro, free tiers elsewhere)	~2 600 Kč	~3 700 Kč	24 950 Kč	~85 %
20	~3 300 Kč (+ Sentry Team, Inngest paid tier, email)	~10 500 Kč	~14 000 Kč	99 800 Kč	~86 %
100	~12 000 Kč (+ larger DB compute, PITR, uptime, analytics)	~42 000–52 000 Kč (volume price)	~55 000–65 000 Kč	499 000 Kč	~88 %
500	~40 000 Kč (+ read replica, higher job volume)	~185 000–260 000 Kč (negotiated 0.7–1.0 Kč)	~225 000–300 000 Kč	2 495 000 Kč	~88–91 %

Consequences for pricing: include an SMS segment allowance per plan (for example Basic 400, Growth 800, Pro 1 500 segments) with overage priced at cost plus margin, and show segment usage in the admin Billing page. The diacritics toggle alone can cut SMS cost by a third to a half. The real scaling constraint is operator time: plan roughly one operator per 80–150 managed shops once automation is in place.

29. Development phases

Each phase ends with a gate you can check; do not start the next phase until the gate is met. Estimates assume one founder working with an AI coding agent.

Phase 0 — Prototype with fake data (1–2 weeks)

Goal: click through the full loop on seed data and show it to 3 shop owners.

- **Build:** repo, Supabase local, schema + RLS for core tables, seed script, admin shell, company list/detail, customer list with filters, segment presets (read-only), campaign builder with AI template and preview, mock SMS send, manual conversion marking, client dashboard with seeded results.
- **Do NOT build:** real SMS, imports of real files, auth polish, billing, alerts, email, automations.
- **Gate:** 3 owners say "I would try this" after a 15-minute demo; you know which export format their garage software produces.

Phase 1 — Internal managed-service MVP (3–5 weeks)

Goal: a secure tool you alone use to run campaigns for shops.

- **Build:** invite-only auth with admin MFA, full import wizard (CSV/XLSX, encoding, mapping, validation, dedup, history), lifecycle recompute job, segment rule engine + presets, campaign builder with holdout, follow-ups, approval flow, real SMS adapter + mock, dispatcher with idempotency, DLR webhooks, opt-out page + suppression, conversion pipeline + manual revenue, client read-only dashboard, admin Overview with Attention Required (first 6 rules), audit log, kill switches, Sentry, backups.
- **Do NOT build:** self-service campaign creation for clients, billing integration, email campaigns, automations, two-way SMS, visit re-import matching (manual first), PostHog.
- **Gate:** all Definition of Done items in section 41 pass; a test campaign to 20 of your own and friends' phones (with consent) runs with zero duplicates and correct opt-out.

Phase 2 — First real shop pilot (6–10 weeks, mostly running, little building)

Goal: prove that contacted customers return more often than the holdout.

- **Build:** visit re-import matching + review queue, monthly results email, owner approval page (mobile), vocative review screen, Fakturoid invoicing by hand, PostHog, remaining alert rules, small fixes from real use.
- **Do NOT build:** public sign-up, Stripe, new channels, automations, integrations, redesigns.
- **Run:** 1–3 shops, 2–3 campaigns each, at least 600 recipients total.
- **Gate:** at least 2 shops with ROI ≥ 3× on attributed revenue, a positive treatment-vs-holdout difference, and at least one shop willing to pay full price.

Phase 3 — Multi-client production SaaS (6–8 weeks)

Goal: 10–30 paying shops, still mostly managed.

- **Build:** plans and limits enforcement, subscriptions with trial, Stripe Billing + Fakturoid API for invoices, self-service mode (client campaign builder behind a flag), client user management, email channel (transactional + marketing), data export for shops, staging environment hardening, PITR backups, status page.
- **Do NOT build:** automations, integrations, A/B testing, AI reply assistant, white label.
- **Gate:** 20 paying shops, churn < 5% per month, operator time < 1 hour per shop per month.

Phase 4 — Automation (6–8 weeks)

Goal: most routine campaigns run without the operator.

- **Build:** automation rules engine (section 13), seasonal tyre and AC rules, auto-send guardrails, opportunity recommendations with confidence, two-way SMS with numeric sender (if chosen), reply inbox with AI-drafted replies, A/B testing of templates, learned per-shop conversion rates.
- **Do NOT build:** churn ML models, integrations, WhatsApp.
- **Gate:** 70% of campaigns created by automations; operator oversees 50+ shops.

Phase 5 — Integrations and scale

Goal: reduce data friction and grow past 100 shops.

- **Build (in demand order):** connectors to the 1–2 most common Czech garage software products among your clients, public API + webhooks, multiple branches, WhatsApp, reseller/white label if a partner channel appears, benchmarking across shops (anonymized, aggregated), review and satisfaction messages if clients ask.
- **Do NOT build:** anything on the "never" list in section 5.
- **Gate:** each integration used by at least 10 shops within 3 months of launch.

30–31. Testing strategy and fake seed data

Test hardest where a bug costs money or trust: tenant isolation, who gets a message, how many times, and how revenue is counted.

30. Testing strategy

Level	Tool	What	Required coverage
Unit	Vitest	Phone normalization, date/money parsing, encoding detection, dedup matching, `compileSegment` per operator, contact rules, sending window + holidays, `countSmsSegments`, template renderer + fallbacks, vocative rules, attribution matching, KPI formulas	90%+ lines in `lib/domain`

Level	Tool	What	Required coverage
Database	pgTAP or SQL tests via Supabase CLI	RLS for every table: A cannot read/write B; manager cannot do owner actions; audit log immutability; composite FK rejects cross-tenant links	Every table, generated from a table list so new tables fail until covered
Integration	Vitest + local Supabase + mock provider	Import end-to-end on fixture files (UTF-8, Windows-1250, `;` delimited, XLSX, broken file); campaign send with retries; DLR out of order; opt-out mid-campaign cancels follow-ups; job re-run creates no duplicates	All happy paths + listed edge cases
E2E	Playwright	F1–F5 flows in section 6 on desktop and a 390 px mobile viewport	Core loop green before every deploy
AI	Vitest with recorded fixtures + validator	Generated templates pass validation; denylist catches bad outputs; prompt version bump requires fixture refresh	20 golden cases
Load	k6 or a script	100,000-row import under 5 minutes; dispatch 5,000 messages respecting rate limit; dashboard p95 < 500 ms with 500 companies of seed data	Before Phase 3
Security	Manual checklist + Supabase advisors + `npm audit`	Headers, secrets in bundle, RLS advisor clean, webhook signature rejection	Before each phase gate

Critical scenarios that must have a named test:

1. A customer in two overlapping campaigns receives only one message thread in 30 days.
2. Re-running the send job after a crash produces zero duplicate messages.
3. A customer who opts out between initial send and day-3 follow-up gets no follow-up.

4. A holdout customer never receives a message and never counts in recovered revenue.
5. A shop manager of company A requesting `/app/customers/<id of company B>` gets 404, and the DB query returns zero rows.
6. A campaign scheduled for Saturday 14:00 moves to Monday 9:00; one scheduled on 28 October (state holiday) moves to the next working day.
7. The mock provider is the only adapter available when `APP_ENV` is not `production`.

31. Fake seed data structure

A deterministic seed (`pnpm db:seed`, fixed Faker seed) builds a realistic world:

Company	Size	Profile
Autoservis Dvořák s.r.o., Kolín	1,800 customers, 2,300 vehicles	Healthy data, 5 years of visits, managed mode, 2 finished campaigns with results
Pneu & Servis Horák, Brno	3,200 customers	Tyre-heavy, many fixed-line phones (warnings), one failed import in history
Garáž U Nádraží, Jihlava	600 customers	Sparse data (no job values, 30% missing vehicles), no campaigns, triggers "no campaign running" alert

Distributions:

- **Brands:** Škoda 35%, Volkswagen 12%, Hyundai 7%, Toyota 6%, Ford 6%, Renault 5%, Peugeot 5%, Kia 5%, Dacia 4%, BMW 3%, Mercedes-Benz 3%, other 9%. Models realistic per brand (Octavia, Fabia, Superb, Kodiaq; Golf, Passat; i30…).
- **Visits per vehicle:** Poisson with mean 4 over 5 years; 25% of vehicles churned (no visit in the last 14+ months), 15% at risk.
- **Service mix and job values (CZK):** regular service 3 500–9 000, oil change 1 800–3 500, tyre change 800–1 600 (spring/autumn peaks), tyre storage 1 000–1 500, AC service 1 500–2 500 (Apr–Jun), brakes 3 000–8 000, diagnostics 800–1 500, STK prep 1 500–4 000, other repairs 2 000–25 000 (long tail).
- **Contacts:** Czech names from Faker `cs_CZ` with correct female surname forms; phones in `+420` format, 90% mobile; emails only on `@example.com`, `@example.cz` domains; 5% with legal basis `unknown`, 3% already suppressed.
- **Plates:** Czech format (digit, region letter, digit/letter, space, four digits, e.g. `2S4 7710`), randomly generated; VIN on 60% of vehicles with valid structure.

- **Messy fixtures** for import tests in `supabase/seed/fixtures/`: a Windows-1250 semicolon CSV with `dd.mm.yyyy` dates and `4 700 Kč` values, an XLSX with two sheets and a summary row, a file with duplicates and malformed phones.

Example seeded record:

```
{
  "customer": { "first_name": "Jana", "last_name": "Procházková",
"phone_e164": "+420 7xx xxx xxx (generated)", "email":
"jana.prochazkova@example.cz", "legal_basis": "existing_customer" },
  "vehicles": [{ "brand": "Škoda", "model": "Octavia",
"license_plate_display": "2S4 7710", "vin": "TMBJJ7NE5L0123456" }],
  "visits": [
    { "visited_at": "2024-09-12", "service_type_raw": "Servis 60 tkm",
"service_category": "regular_service", "job_value_minor": 612000 },
    { "visited_at": "2025-04-03", "service_type_raw": "Přezutí + uskladnění",
"service_category": "tyre_change", "job_value_minor": 140000 }
  ]
}
```

Seed phones are generated but still real-format, which is why non-production environments can only use the mock SMS adapter. Seeded users: `admin@example.com` (super admin, MFA disabled in local only), one owner and one manager per company, password printed by the seed script.

32–34. Deployment, backup and recovery, scaling

Three environments, migrations applied by CI only, independent backups you can restore without the vendor, and scaling by indexes and precomputed stats long before any architectural change.

32. Deployment strategy

Environment	Database	SMS	Deploys from	Data
Local	Supabase CLI (Docker)	mock	developer machine	Seed only
Staging	Separate Supabase project (EU)	mock (real provider only with a whitelist of your own phones)	`main` branch, automatically	Seed only
Production	Supabase Pro project (EU)	real	Git tag `v*`, manual approval	Real

Vercel preview deployments for pull requests point to staging, never production. Production environment variables are scoped to the Production environment in Vercel.

CI (GitHub Actions) on every PR: install, typecheck, lint, unit tests, spin up local Supabase, apply migrations from scratch, run DB/RLS tests, integration tests, Playwright smoke tests, build, bundle secret scan, RLS-enabled check.

Release: tag → CI applies migrations to production with `supabase db push` → Vercel promotes the build → Inngest syncs functions → smoke test hits `/api/health`. Migrations must be backward compatible with the previous app version (expand, deploy, contract), so a rollback of the app never requires a rollback of the schema.

Feature flags in `feature_flags` gate risky features per company (self-service, auto-send, vocatives).

33. Backup and recovery

- **Supabase daily backups** (included in Pro, 7-day retention).
- **Point-in-time recovery** add-on from Phase 3 (RPO minutes).
- **Independent nightly logical dump** (`pg_dump`, custom format) by a scheduled GitHub Action or small worker, encrypted with age/GPG, uploaded to EU object storage in another provider's account (e.g. Cloudflare R2 with EU jurisdiction, or S3 `eu-central-1`), 30-day retention with lifecycle rules. Protects against account-level loss.
- **Storage bucket:** import files are temporary by design and not backed up; logos synced weekly.
- **Targets:** Phase 1–2 RPO 24 h, RTO 8 h. Phase 3+ RPO 15 min, RTO 4 h.
- **Restore drill** every quarter: restore the latest dump into a fresh project, run the RLS test suite and a row-count comparison, log the result.
- **Deleted data and backups:** erasure requests are not applied to backups; backups age out within 35 days, and a restore re-applies erasures from the `data_subject_requests` log before going live.
- **Incident runbook** in the repo (`docs/runbooks/`): provider outage, duplicate send, data leak suspicion, failed migration, restore from dump.

34. Scaling strategy

Clients	Expected load	Actions
1–20	< 10k messages/month, < 100k vehicles	Nothing special. Indexes from section 18 are enough

Clients	Expected load	Actions
20–100	~50k messages/month, ~500k vehicles	`company_monthly_stats` for dashboards; nightly jobs fan out per company with concurrency limits; bump Supabase compute one size
100–500	~250k messages/month, a few million visits	Partition `messages` and `message_events` by month; archive message bodies > 24 months; read replica for admin analytics; per-provider dispatch concurrency; second SMS provider for failover
500–1,000+	~1M messages/month	Dedicated job worker if Inngest costs grow; consider moving heavy nightly segment evaluation to SQL functions run in the database; connection pooling via Supavisor in transaction mode everywhere

Rules that keep scaling cheap: never compute dashboards from raw messages on page load; always filter by `company_id` first; batch everything (500-row upserts, 100-message dispatch batches); keep job payloads to IDs.

35–36. Technical debt risks and what not to overengineer

The expensive debts here are data-model and sending-safety shortcuts; almost everything else can be cheaply fixed later.

35. Technical debt risks

Risk	Why it hurts later	Prevention now
Flat customer table without vehicles and visits	Rebuilding segments and attribution means a data migration for every shop	Vehicle-centric model from day one
Business logic inside React components or route handlers	Automations and integrations need the same logic from jobs	All domain logic in `lib/domain`, framework-free, unit-tested
Service-role queries without company filter	One missed `where` leaks data across shops	`db.forCompany()` helper, lint rule, composite FKs

Risk	Why it hurts later	Prevention now
Non-idempotent sends	Duplicate SMS erodes trust immediately	Unique idempotency keys, `sending` state review
Mutable revenue totals	Numbers drift and can't be explained	Derive from conversions; stats rebuildable
Hardcoded Czech strings	Painful i18n retrofit	next-intl from the first component
Provider-specific fields in core tables	Provider switch becomes a migration	Provider data in `message_events.payload` and adapter code
Settings scattered across columns	Every new threshold is a migration	One Zod-validated `companies.settings` JSON with defaults in code
AI prompts inline in code	Can't compare or roll back behaviour	Versioned prompt files in `lib/ai/prompts/`, version logged per generation
No holdout from the start	Early results can never be validated retroactively	Holdout in the first real campaign
Letting the agent add dependencies freely	Bloat and supply-chain risk	Dependency list fixed in the prompt; new ones need a reason

36. What NOT to overengineer

- **Roles and permissions:** a TypeScript map and three roles. No policy engine, no custom role builder.
- **Segment builder UI:** presets + a simple rule form. No drag-and-drop query builder.
- **Automation engine:** daily cron over segments. No event bus, no workflow designer canvas.
- **Multi-tenancy:** shared schema + RLS. No schema-per-tenant, no sharding.
- **Analytics:** one monthly stats table and a few charts. No warehouse, no BI tool, no custom report builder.
- **AI:** one template call and four rewrite actions. No agents, no fine-tuning, no embeddings, no chatbot.
- **Billing:** manual invoices until Phase 3. No proration logic, no usage-based metering engine.
- **Design system:** shadcn/ui as-is plus a few tokens. No Storybook in MVP.

- **Infrastructure:** managed services only. No Docker in production, no Terraform until a second engineer joins.
- **Notifications:** in-app list + email. No push notifications, no notification preference matrix beyond on/off per type.
- **Integrations:** CSV import done well is the integration layer for the first 50 shops.

37–39. Folder structure, API architecture, environment variables

One Next.js app, one `lib/domain` core that knows nothing about Next.js, and three entry points into it: server actions for the UI, route handlers for external callers, Inngest functions for background work.

37. Project structure

```
.
├── app/
│   ├── (auth)/login, reset-password, accept-invite
│   ├── (client)/app/                      # shop UI: page.tsx (Přehled),
customers/, campaigns/, results/, settings/
│   ├── (admin)/admin/                     # overview,
companies/[companyId]/…, campaigns/, alerts/, billing/, system/
│   ├── o/[token]/page.tsx                 # public opt-out page
│   └── api/
│       ├── inngest/route.ts               # Inngest handler
│       ├── webhooks/sms/[provider]/route.ts
│       ├── health/route.ts
│       └── (later) v1/…                   # public API, Phase 5
├── components/
│   ├── ui/                                # shadcn primitives
│   ├── dashboard/  campaigns/  customers/  imports/  admin/
├── lib/
│   ├── domain/                            # pure business logic, no Next.js
imports
│   │   ├── lifecycle/  segments/  campaigns/  contact-rules/  attribution/
│   │   ├── imports/  (parse, normalize, validate, dedupe)
│   │   ├── messaging/ (render, segments, sending-window)
│   │   └── alerts/   (health rules)
│   ├── providers/
│   │   ├── sms/  (types.ts, mock.ts, <provider>.ts, index.ts)
│   │   └── email/
│   ├── ai/  (client.ts, prompts/*.v1.ts, validate.ts)
│   ├── db/  (server-client.ts, service-client.ts, for-company.ts,
types.gen.ts)
│   ├── auth/  (context.ts, permissions.ts, mfa.ts)
```

```
|   ├── actions/                          # server actions, thin: validate → assertCan → domain → audit
|   ├── jobs/                             # Inngest functions: import, lifecycle, campaign-send, dispatch, dlr, stats, alerts, retention
|   ├── audit.ts  errors.ts  money.ts  dates.ts  i18n.ts  rate-limit.ts  env.ts
├── messages/ cs.json  en.json
├── supabase/
|   ├── migrations/                       # timestamped SQL
|   ├── tests/                            # pgTAP / RLS tests
|   ├── seed/ (seed.ts, fixtures/)
|   └── config.toml
├── tests/ unit/  integration/  e2e/
├── docs/ (spec.md, decisions/ADR-*.md, runbooks/)
├── .github/workflows/ ci.yml, release.yml, backup.yml
├── CLAUDE.md / AGENTS.md                 # rules for coding agents (from section 42)
└── .env.example
```

38. API architecture

Entry point	Used for	Auth	Rules
Server Components	Reads for pages	User session, RLS	Query via `lib/db/server-client`, never the service client
Server Actions (`lib/actions`)	All UI mutations	`requireContext({ permission, companyId })`	Zod input, domain call, audit, typed result object
Route handlers `/api/webhooks/*`	Provider callbacks	Signature/secret	Store raw event, enqueue job, return 200 in < 1 s
Route handler `/api/inngest`	Job runner	Inngest signing key	Only registered functions
Route handler `/api/health`	Uptime	Public, no data	DB ping + provider status, cached 60 s
Page `/o/[token]` + action	Opt-out	Token only	Rate-limited per IP; POST to unsubscribe
Public REST `/api/v1`	Integrations, Phase 5	API keys per company (hashed), scopes	Versioned, OpenAPI documented

Main domain events (emitted as Inngest events, also used for the audit trail): `import/uploaded`, `import/commit.requested`, `campaign/scheduled`, `campaign/step.due`, `message/queued`, `message/dlr.received`, `customer/opted_out`, `conversion/completed`, `stats/recompute.requested`. Payloads carry IDs only.

39. Environment variables

```
# App
APP_ENV=local                          # local | staging | production
NEXT_PUBLIC_APP_URL=http://localhost:3000
OPT_OUT_BASE_URL=http://localhost:3000/o

# Supabase
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=             # server + jobs only
SUPABASE_DB_URL=                       # migrations / backups only, CI secret
```

```
# Jobs
INNGEST_EVENT_KEY=
INNGEST_SIGNING_KEY=

# Messaging
SMS_PROVIDER=mock                    # mock | <provider id>; non-production
forces mock
SMS_PROVIDER_API_KEY=
SMS_PROVIDER_API_SECRET=
SMS_WEBHOOK_SECRET=
SMS_TEST_WHITELIST=                  # staging: comma-separated E.164
numbers allowed for real sends
EMAIL_PROVIDER_API_KEY=
EMAIL_FROM="Návrat <no-reply@example.com>"

# AI
AI_PROVIDER=openai                   # openai | anthropic
AI_MODEL=
AI_API_KEY=

# Security
SUPPRESSION_PEPPER=                  # 32+ random bytes, base64; never
rotate without re-hashing
UPSTASH_REDIS_REST_URL=
UPSTASH_REDIS_REST_TOKEN=

# Monitoring
SENTRY_DSN=
NEXT_PUBLIC_SENTRY_DSN=
SENTRY_AUTH_TOKEN=                   # CI only, source maps
NEXT_PUBLIC_POSTHOG_KEY=             # Phase 2

# Billing (Phase 3)
FAKTUROID_ACCOUNT_SLUG=
FAKTUROID_CLIENT_ID=
FAKTUROID_CLIENT_SECRET=
STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=

# Backups (CI)
BACKUP_BUCKET_URL=
BACKUP_ACCESS_KEY_ID=
BACKUP_SECRET_ACCESS_KEY=
BACKUP_ENCRYPTION_PUBLIC_KEY=
```

`lib/env.ts` validates all variables with Zod at boot and refuses to start if `APP_ENV != production` and `SMS_PROVIDER != mock` without `SMS_TEST_WHITELIST`.

40–41. Implementation order and MVP Definition of Done

Build bottom-up in 16 slices; each slice ends green in CI and is demoable on seed data before the next one starts.

40. Step-by-step implementation order

1. **Foundation:** Next.js + TS strict, Tailwind, shadcn/ui, next-intl (cs), ESLint/Prettier, Vitest, Playwright, `lib/env.ts`, GitHub Actions CI, local Supabase.
2. **Schema v1 + RLS:** tenancy, customers, vehicles, visits, audit logs; RLS helper functions; RLS test harness generated from the table list; RLS-enabled CI check.
3. **Auth:** login, reset, invitations, `requireContext`, permissions map, admin MFA, route groups and layouts for `/app` and `/admin`.
4. **Seed:** deterministic Faker seed for 3 companies (section 31) and import fixtures.
5. **Admin companies:** create/edit company, settings JSON with Zod defaults, DPA confirmation, sending switch, invite owner.
6. **Customers:** list with filters and search, customer detail with vehicles and visit timeline.
7. **Import wizard:** upload to storage, Inngest parse job (encoding, delimiter, XLSX), mapping UI with saved templates, validation, review, commit job, history, error CSV. AI mapping hint last.
8. **Lifecycle job:** nightly + post-import recompute of status and eligibility; service category mapping UI.
9. **Segments:** DSL, `compileSegment`, presets, contact rules, eligible count preview with exclusion reasons.
10. **Campaigns (no sending):** builder steps, holdout selection, templates with placeholders, renderer, segment counter, previews, expected revenue, status machine, approval flow.
11. **AI template generation:** provider wrapper, versioned prompt, validator, rewrite actions, `ai_generations` log, vocative rules + review.
12. **Messaging:** provider interface, mock adapter, dispatcher with idempotency and rate limit, sending window + holidays, pre-send checks, kill switches, follow-up scheduling with stop conditions.
13. **Opt-out + DLR:** tokens, public page, suppression list, webhook route, event processing, real provider adapter (tested on staging whitelist only).

14. **Conversions:** pipeline buttons on customer and campaign pages, revenue entry, `company_monthly_stats` job, attribution window closing job.
15. **Dashboards:** client Přehled and Výsledky, admin Overview, clients list with health, Attention Required with first 6 rules, daily digest email.
16. **Hardening:** security headers, rate limits, Sentry, backups job, retention job, restore drill, E2E suite, runbooks, production deploy.

41. Definition of Done for the MVP

Functional

- [ ] From the admin account you can create 3 shops, import a messy Windows-1250 CSV and an XLSX for each, and see correct customer, vehicle and visit counts.
- [ ] Invalid, warning and duplicate rows are shown with reasons; re-importing the same file creates no duplicates.
- [ ] Lifecycle statuses and all preset segments return the counts expected by fixture tests.
- [ ] You can create a campaign from a preset, generate an AI template, edit it, see the segment count and cost, preview 5 recipients, add 2 follow-ups, and schedule it.
- [ ] Approval flow works in both modes (operator-only and client approval).
- [ ] A real SMS campaign to the staging whitelist sends, receives delivery reports, and follow-ups stop after opt-out.
- [ ] The opt-out link works on mobile in one tap, is idempotent, and survives customer deletion.
- [ ] Holdout recipients receive nothing and appear in the holdout comparison.
- [ ] Marking booked/completed with revenue updates the client dashboard within 1 minute.
- [ ] Client dashboard shows recovered revenue, 5 KPI cards, monthly chart, campaigns, recent returns and opportunities, readable on a 390 px phone.
- [ ] Attention Required shows campaign failure, import needing review, approval waiting, high SMS failure rate, no campaign running and low conversion, and shows "Vše běží normálně" when none apply.

Safety and quality

- [ ] RLS tests pass for every table; a shop user cannot access another shop's data through UI, server actions or direct Supabase queries.
- [ ] Killing the dispatcher mid-send and re-running it produces zero duplicate messages.
- [ ] Messages never go out outside the sending window or on Czech public holidays.

- [ ] Non-production environments cannot reach the real SMS provider except whitelisted numbers.
- [ ] Every action in section 23 writes an audit entry; audit rows cannot be updated or deleted.
- [ ] Admin support view is read-only by default, shows the banner and logs the reason.
- [ ] No phone numbers or emails appear in Sentry events, logs or AI requests.
- [ ] Nightly independent backup exists and one restore drill has been completed.
- [ ] Typecheck, lint, unit, DB, integration and E2E suites are green in CI.
- [ ] Signed DPA template and sub-processor list exist; legal review of the messaging and consent approach is done.

42. Coding-agent prompt for Phase 1

Export this doc as Markdown into `docs/spec.md` in an empty repository, then give the agent the prompt below. It works as the first message in Claude Code or Cursor and as the content of `CLAUDE.md` / `AGENTS.md`.

```
# Role
You are the lead engineer building "Návrat", a multi-tenant B2B SaaS that
helps Czech car repair shops win back inactive customers with SMS campaigns
and shows the revenue recovered. The full product specification is in
docs/spec.md. Read it completely before writing code. When this prompt and
the spec disagree, this prompt wins; when the spec is silent, choose the
simplest secure option and record it in docs/decisions/ADR-XXX.md.

# Goal of this engagement
Build Phase 1 (spec section 29): an internal, invite-only tool the operator
(super admin) uses to run campaigns for 1–5 shops, plus a read-only client
dashboard. Follow the implementation order in spec section 40, one slice at a
time. The finish line is the Definition of Done in spec section 41.

# Stack (do not change without asking)
- Next.js (App Router) + TypeScript strict, pnpm
- Tailwind CSS + shadcn/ui, Recharts, TanStack Table, react-hook-form + Zod
- Supabase: Postgres (EU), Auth (TOTP MFA for admins), Storage; supabase-js
with @supabase/ssr; SQL migrations via Supabase CLI; generated types
- Inngest for all background jobs
- next-intl, Czech UI strings in messages/cs.json; all code, identifiers,
comments and docs in English
- Vitest, Playwright, SQL/pgTAP tests for RLS
- Papa Parse, SheetJS (official CDN tarball), iconv-lite, libphonenumber-js,
date-fns, @faker-js/faker (cs_CZ)
- Vercel AI SDK with a provider set by env (AI_PROVIDER, AI_MODEL)
```

```
- Sentry, Upstash rate limiting
Ask before adding any other dependency, and state why it is needed.

# Non-negotiable rules
1. Tenant isolation: every tenant table has company_id, RLS enabled and
forced, policies using app.company_role() / app.is_platform_admin(), and
composite foreign keys (id, company_id). Add RLS tests for every new table in
the same change. CI must fail if any public table lacks RLS.
2. The service-role client is used only in jobs and server-only modules,
always through db.forCompany(companyId). Never import it in client
components.
3. Business logic lives in lib/domain as pure, framework-free TypeScript with
unit tests. Server actions are thin: parse input with Zod, requireContext({
permission, companyId }), call domain, write audit(), return a typed result.
4. Sending safety: messages get a unique idempotency_key; the dispatcher
claims rows with FOR UPDATE SKIP LOCKED; rows stuck in 'sending' go to manual
review, never auto-retry; suppression and stop conditions are re-checked
right before each send; sending window Mon–Fri 9–18, Sat 9–12, Europe/Prague,
never Sundays or Czech public holidays; global and per-company kill switches.
5. Non-production environments may use only the mock SMS provider, except
numbers in SMS_TEST_WHITELIST. Enforce this in lib/env.ts at boot.
6. Every message includes the opt-out placeholder; the renderer refuses to
send without it. Opt-out happens by POST on /o/[token].
7. Customers with legal_basis 'unknown', fixed-line phones, suppression hits
or contact-cap conflicts are never eligible.
8. Every campaign has a holdout group (default 10%) that receives no messages
and never counts as recovered revenue.
9. AI never receives customer personal data (names, phones, emails, plates,
notes). AI writes templates with placeholders; code renders them. Validate AI
output per spec section 15.
10. No real personal data anywhere outside production. Seed data only; emails
on example.com / example.cz.
11. Money is stored as bigint haléře (*_minor) and formatted with
Intl.NumberFormat('cs-CZ', { style: 'currency', currency: 'CZK',
maximumFractionDigits: 0 }). Dates display as "25. 9. 2026".
12. No personal data in logs, Sentry events or job payloads (IDs only).
13. Audit every action listed in spec section 23. audit_logs is append-only
(trigger blocks update/delete).

# Out of scope for Phase 1 (do not build, even partially)
Public sign-up, Stripe or any billing integration, email campaigns,
automation rules engine, two-way SMS or reply inbox, garage software
integrations, A/B testing, WhatsApp, native apps, client-side campaign
creation (self-service), visit re-import matching (Phase 2). Keep the schema
ready where the spec says so, nothing more.
```

```
# UX direction
Premium but plain B2B dashboard in the style of Stripe or Linear: white
background, neutral grays, one accent (#0F7B53) for money and positive
states, Inter with tabular numerals, 8 px spacing grid, shadcn/ui primitives
as-is. No gradients, no AI sparkle aesthetics, max six sidebar items. On the
client dashboard, "Vrácené tržby tento měsíc" is the largest element on the
screen. Everything must work at 390 px width.

# How to work
- Work slice by slice from spec section 40. Before each slice, post a short
plan (files, migrations, tests). After each slice: run typecheck, lint, all
tests; fix failures; summarize what changed, what you assumed, and how to
demo it on seed data. Then stop and wait for "continue".
- Write migrations as timestamped SQL in supabase/migrations. Never edit an
applied migration; add a new one. Keep migrations backward compatible (expand
→ migrate → contract).
- Prefer boring, explicit code over abstractions. No generic repository
layers, no event bus, no custom component library.
- When unsure about a legal or business rule, implement the stricter
behaviour and flag it in your summary.

# First session
Complete slices 1–4 of spec section 40:
1. Project foundation, CI, env validation.
2. Schema v1 (companies, profiles, memberships, invitations, platform_admins,
customers, vehicles, visits, service_category_mappings, audit_logs) with RLS
helper functions, policies, composite FKs, and the generated RLS test
harness.
3. Auth: login, password reset, invitation acceptance, admin MFA,
requireContext, permissions map, /app and /admin layouts with sidebars and
Czech labels.
4. Deterministic seed for the 3 companies in spec section 31, plus the import
fixtures.
Demo target: log in as the seeded super admin, see the companies list, open
one company, see its customers; log in as a seeded shop manager and confirm
you can see only your own company's customers.
```